

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INGENIERIA EN SOFTWARE



2
0
2
6

Estudiantes:

Andrade Danna, Lainez Ricardo,
Santi Jeancarlo, Tufiño Erick

Asignatura: Desarrollo de Aplicaciones Móviles

Tema: Informe Sistema de Reservas HostSigchos

NRC: 30738

Docente: Doris Karina Chicaiza Angamarca

Fecha: 25 de julio de 2026

Índice general

Resumen ejecutivo	1
1. Tema, introducción y objetivos	2
1.1. Tema	2
1.2. Introducción	2
1.3. Planteamiento del problema	3
1.4. Objetivo general	3
1.5. Objetivos específicos	3
1.6. Alcance	4
2. Marco Conceptual y Trabajos Relacionados	5
2.1. Trabajos relacionados	5
2.2. Conceptos Clave	6
2.2.1. Overbooking (Sobreventa)	6
2.2.2. Clean Architecture y MVVM	6
2.2.3. Transacciones Atómicas	6
2.2.4. Geolocalización Operativa y Magnetometría	6
2.2.5. Inteligencia Artificial Generativa y NLP	6
2.2.6. Sistemas Reactivos y WebSockets	6
2.2.7. Modelo de Seguridad Zero Trust y JWT	7
2.2.8. Entornos de Ejecución Confiables (TEE)	7
2.2.9. Ofuscamiento y Mitigación de Ingeniería Inversa	7
3. Arquitectura y Estructura	8
3.1. Arquitectura del Sistema	8
3.1.1. Capas de Arquitectura Limpia	8

3.1.2. Inversión de Dependencias (GetIt)	8
3.1.3. Diagrama de Despliegue	9
3.2. Stack Tecnológico, Herramientas y APIs Integradas	10
3.3. Diagrama de carpetas	11
3.4. Modelo de datos	12
3.5. Diagramas UML del Sistema	14
3.5.1. Diagrama de Componentes (Clean Architecture)	14
3.5.2. Diagrama de Clases del Dominio	14
3.5.3. Diagrama de Casos de Uso	15
3.5.4. Diagrama de Secuencia (o Actividad)	17
3.5.5. Máquina de Estados de Reserva	17
3.5.6. Diagrama Entidad-Relación Extendido (ER)	18
4. Desarrollo y capturas del aplicativo	20
4.1. Módulo Web y Entorno Legal	20
4.2. Módulo de Autenticación, Registro y Biometría	23
4.3. Módulo de Exploración (Dashboard) y Visualización de Hosterías	27
4.4. Módulo de Mapas Interactivos y Geolocalización	32
4.5. Asistente Virtual (Chatbot con Inteligencia Artificial)	33
4.6. Proceso de Reserva y Resolución del Overbooking	35
4.7. Módulo de Usuario, Ajustes y Notificaciones	38
4.8. Panel Administrativo Web	42
4.9. Seguridad y Privacidad	43
5. Pruebas	45
5.1. Estrategia general	45
5.2. Entorno Tecnológico, Herramientas y Guía de Ejecución	45
5.2.1. Herramientas de QA y Tecnologías Empleadas	45
5.2.2. Protocolo y Procedimientos Paso a Paso de Ejecución	46
5.3. Pruebas de usabilidad	47
5.3.1. Participantes	47
5.3.2. Métricas	47
5.4. Pruebas funcionales	48
5.5. Pruebas de Red (Rate Limiting)	49

5.6. Pruebas de rendimiento y carga	50
5.7. Niveles de Pruebas Automatizadas en Código	51
5.7.1. Pruebas Unitarias (Lógica de Dominio y Datos)	51
5.7.2. Pruebas de Interfaz In-Memory (Widget Testing)	54
5.7.3. Pruebas de Integración (End-to-End)	55
5.7.4. Pruebas Funcionales Específicas	58
5.7.5. Pruebas de Seguridad y Throttling (Rate Limit)	59
5.7.6. Pruebas de Rendimiento y Latencia (CPU/GPU)	59
5.7.7. Pruebas de Automatización Híbrida (Appium)	60
6. Conclusiones	63
7. Recomendaciones	64

Índice de figuras

1.1. Logotipo oficial de la aplicación móvil HostSigchos.	2
3.1. Flujo de datos bidireccional desde la capa de Presentación hasta el Origen de Datos mediado por los Casos de Uso.	9
3.2. Diagrama de Despliegue mostrando la topología física y los protocolos de comunicación.	10
3.3. Estructura documental y referencias simuladas (Foreign Keys) en Cloud Firestore.	13
3.4. Diagrama de Componentes de la Arquitectura.	14
3.5. Diagrama de Clases de las Entidades de Dominio.	15
3.6. Diagrama de Casos de Uso general del sistema HostSigchos.	16
3.7. Diagrama UML del flujo de reservas y validación.	17
3.8. Diagrama de Estados para el ciclo de vida transaccional.	18
3.9. Diagrama ER lógico de las colecciones de Firestore.	19
4.1. Landing Page promocional del ecosistema HostSigchos.	21
4.2. Visualización de las Políticas de Privacidad.	22
4.3. Documento legal de Términos y Condiciones.	23
4.4. Pantalla de inicio de sesión clásica y SSO (Google).	24
4.5. Pantalla de registro con validaciones de formulario en tiempo real.	25
4.6. Módulo de recuperación de contraseña vía correo electrónico.	26
4.7. Autenticación biométrica habilitada por el hardware nativo.	27
4.8. Dashboard principal de exploración en formato Lista.	28
4.9. Dashboard principal de exploración alternado al formato Cuadrícula.	29
4.10. Vista detallada de los servicios generales de una hostería.	30
4.11. Módulo comunitario de calificación y reseñas.	31
4.12. Catálogo detallado de habitaciones y precios por noche.	32

4.13. Pantalla de visualización de mapas integrando los marcadores de hosterías y geolocalización.	33
4.14. Asistente turístico integrado, respondiendo con texto y voz.	34
4.15. Módulo centralizado de gestión y solicitud de nueva reserva.	35
4.16. Selector de rango de fechas para validación de disponibilidad.	37
4.17. Proceso de <i>Checkout</i> detallando costos antes del guardado final.	38
4.18. Interfaz de configuración y edición del perfil del turista.	39
4.19. Flujo de seguridad para la eliminación definitiva de la cuenta.	40
4.20. Centro de Ayuda y Soporte técnico al cliente.	41
4.21. Bandeja de entrada de Notificaciones Push sobre el estado de las reservas. . . .	42
4.22. Tablero de control web para gestión y actualización de estado de reservas. . . .	43
5.1. Captura de pantalla de la terminal validando el manejo del Rate Limit.	50
5.2. Evidencia gráfica del pico de tráfico en Firebase (Prueba de estrés PASADA). . . .	50
5.3. Reporte de consola certificando la suite de ChatMessage Entity.	52
5.4. Reporte de consola certificando la suite de Habitacion Entity.	52
5.5. Reporte de consola certificando la suite de Hosteria Entity.	52
5.6. Reporte de consola certificando la suite de NotificacionApp Entity.	52
5.7. Reporte de consola certificando la suite de Resena Entity.	53
5.8. Reporte de consola certificando la suite de Reserva Entity.	53
5.9. Reporte de consola certificando la suite de Usuario Entity.	53
5.10. Reporte de consola validando el caso de uso CrearReservaUseCase.	53
5.11. Reporte de consola validando el caso de uso VerificarDisponibilidadUseCase. . .	54
5.12. Reporte de ejecución de la suite de pruebas GradientButton Widget.	54
5.13. Reporte de ejecución de la suite de pruebas LoadingOverlay Widget.	55
5.14. Reporte de ejecución de la suite de pruebas CustomTextField Widget.	55
5.15. Reporte de integración MultiProvider de la pantalla HomeScreen.	55
5.16. Reporte de integración con ViewModels de la pantalla LoginScreen.	55
5.17. Reporte oficial de la terminal certificando los 15 flujos de interacción E2E superados exitosamente.	58
5.18. Captura de pantalla de la terminal certificando la ejecución exitosa de las pruebas funcionales.	59
5.19. Captura de pantalla evidenciando el bloqueo (HTTP 429) y protección del sistema contra desbordamientos.	59

5.20. Métricas de rendimiento en consola validando el renderizado de gráficos a 60 FPS ininterrumpidos.	59
5.21. Captura de pantalla de la terminal ejecutando las pruebas con Appium y Node.js.	61
5.22. Ejecución exitosa de la suite de pruebas completas (Unitarias, Widget e Integración E2E).	62
5.23. Monitoreo de estabilidad de red y transacciones exitosas en la base de datos.	62

Índice de tablas

2.1. Relación entre trabajos relacionados y decisiones del proyecto.	5
5.1. Métricas de usabilidad aplicadas en HostSigchos.	48
5.2. Resultados tabulados de la evaluación de usabilidad y satisfacción general. . .	48
5.3. Casos de prueba funcionales principales.	48
5.4. Casos de prueba End-to-End (E2E) completos.	56

Resumen ejecutivo

HostSigchos es una solución integral para la gestión de reservas de hosterías en el cantón Sigchos, Cotopaxi. El sistema moderniza el flujo de reservaciones mediante una aplicación móvil Flutter (para los turistas) y un panel administrativo web React (para los propietarios), respaldados por un backend en Firebase. El objetivo del proyecto es mitigar problemáticas críticas como el *overbooking* (sobreventa) y la falta de digitalización en el sector turístico de la zona.

El presente documento aborda detalladamente el planteamiento del problema, los objetivos generales y específicos, y el alcance. Describe la arquitectura del sistema implementada bajo los principios de **Clean Architecture** y el patrón **MVVM**, incluye el diagrama de carpetas completo y explica el modelo de datos no relacional alojado en **Cloud Firestore**. Finalmente, presenta un recorrido funcional con capturas del aplicativo y documenta la evidencia de pruebas de validación, concluyendo con el impacto tecnológico de la solución.

1 Tema, introducción y objetivos

1.1 Tema

Implementación y documentación de **HostSigchos**, un sistema móvil y web para la gestión automatizada de reservaciones turísticas de hosterías en el cantón Sigchos, integrando disponibilidad en tiempo real, inteligencia artificial, mapas interactivos, hardware del dispositivo y administración centralizada mediante arquitecturas limpias y Firebase.

1.2 Introducción



Figura 1.1: Logotipo oficial de la aplicación móvil HostSigchos.

El cantón Sigchos, ubicado en la provincia de Cotopaxi, posee un inmenso atractivo turístico gracias a su proximidad con el cráter del Quilotoa y sus paisajes andinos. Sin embargo, su infraestructura de servicios hoteleros carece de una digitalización adecuada, lo que limita su capacidad para atraer turistas modernos que esperan procesos ágiles y automatizados [1].

“HostSigchos” nace como una solución tecnológica integral que cierra esta brecha. Está compuesto por una aplicación móvil multiplataforma desarrollada en Flutter para los turistas, que les permite explorar hosterías, verificar disponibilidad de habitaciones con cruce de fechas, chatear con un asistente virtual y confirmar reservas. A su vez, se ha desarrollado un panel de administración web en React JS para que los propietarios gestionen su infraestructura,

precios y el estado de sus reservas. El ecosistema completo está orquestado mediante Firebase, utilizando bases de datos en tiempo real (Firestore) y un sólido manejo de autenticación biométrica y tradicional.

El código fuente completo de la solución puede ser consultado en el siguiente repositorio oficial: <https://github.com/Dval05/HostSigchos-AppMovil>.

1.3 Planteamiento del problema

Actualmente, la gestión de reservaciones en la gran mayoría de las hosterías del cantón Sigchos se realiza a través de métodos convencionales y empíricos. Las reservaciones se toman mediante llamadas telefónicas, mensajes de WhatsApp o agendas físicas. Esto genera una serie de problemáticas graves:

- **Sobreventa (Overbooking):** La falta de sincronización en tiempo real causa que múltiples clientes puedan reservar la misma habitación para las mismas fechas de forma simultánea.
- **Gestión ineficiente de tarifas:** Los propietarios no pueden actualizar dinámicamente sus precios en temporadas altas o bajas de forma centralizada y transparente.
- **Falta de visibilidad digital:** Los turistas externos no tienen un catálogo centralizado donde comparar opciones, leer reseñas de otros huéspedes o verificar los servicios disponibles con geolocalización.
- **Fricción en el proceso de reserva:** El proceso manual requiere que el cliente espere la confirmación humana y envíe comprobantes que muchas veces se traspapelan.

1.4 Objetivo general

Desarrollar, implementar y documentar un sistema integral de gestión de reservas turísticas (Aplicación Móvil + Panel Web) para el cantón Sigchos, que automatice el control de disponibilidad de habitaciones, integre autenticación biométrica, y sincronice datos en tiempo real mediante arquitectura Clean Architecture, Provider y los servicios en la nube de Firebase.

1.5 Objetivos específicos

- OE1.** Aplicar el patrón **Clean Architecture** y **MVVM** en la aplicación móvil (Flutter) empleando **Provider** como gestor de estados para separar la interfaz gráfica de la lógica de negocio y el acceso a datos.
- OE2.** Implementar el diseño de base de datos en **Cloud Firestore** relacionando usuarios, hosterías, habitaciones y reservas, evitando el solapamiento de fechas al momento de realizar transacciones asíncronas.

- OE3.** Integrar funcionalidades de hardware móvil avanzadas como la ubicación (GPS), la cámara (captura de avatares), brújula y lectura biométrica.
- OE4.** Construir un panel web en React que funcione como punto administrativo para los dueños de las hosterías.
- OE5.** Desplegar las aplicaciones (APK en el caso móvil y plataforma en la nube para el frontend web) realizando las pruebas pertinentes funcionales y unitarias.

1.6 Alcance

El ecosistema HostSigchos abarca de forma completa las necesidades tecnológicas para llevar un control estricto y en tiempo real de los arrendamientos turísticos. Su alcance involucra tres perfiles funcionales, divididos en dos plataformas:

1. **Turistas (App Móvil Flutter):** Tienen acceso al catálogo de hosterías (filtrable y con detalle de reseñas), geolocalización utilizando mapas interactivos con Leaflet, interacción con IA para obtener recomendaciones climáticas y sugerencias del entorno, flujo transaccional de carrito y confirmación de la reserva.
2. **Propietarios (Dashboard Web React):** Cuentan con un panel administrativo web desplegado en Vercel, el cual les permite autorizar nuevas reservas (cambio de estado), deshabilitar inventario de habitaciones e intervenir en caso de inconsistencias, controlando el backend en tiempo real.
3. **Módulos transaccionales y de hardware:** El alcance incorpora validaciones biométricas al momento de iniciar sesión, uso de la brújula y el geolocator en el módulo de mapas y la posibilidad de cambiar idiomas y temas dentro de la aplicación móvil de manera reactiva.

2 Marco Conceptual y Trabajos Relacionados

2.1 Trabajos relacionados

Los sistemas de reservaciones hoteleras y turísticas han evolucionado hacia la digitalización mediante ecosistemas web y móviles. Investigaciones recientes destacan la importancia de centralizar inventarios para prevenir la sobreventa. Por ejemplo, Satanasawapak et al. proponen el uso de validaciones en tiempo real mediadas por la nube para mejorar la seguridad transaccional [1]. A diferencia de los sistemas tradicionales, HostSigchos no solo permite la visualización del inventario, sino que delega la validación de solapamiento temporal matemáticamente a la lógica de negocio local (Clean Architecture) antes de enviar la transacción a Firebase, optimizando el ancho de banda y la velocidad de respuesta.

Tabla 2.1: Relación entre trabajos relacionados y decisiones del proyecto.

Enfoque previo	Hallazgo principal	Aplicación en HostSigchos
Arquitecturas en capas [2]	Separación de la UI y la lógica de negocio para mantenibilidad.	Uso estricto de Clean Architecture y MVVM con Provider .
Bases de datos en tiempo real	Sincronización inmediata para evitar discrepancias de estado.	Cloud Firestore con WebSockets y Transacciones atómicas.
Inteligencia artificial en turismo	Personalización y asistencia al viajero 24/7.	Integración de Gemini API para un Chatbot interactivo.
Sistemas reactivos	Actualización de interfaces sin recargar la pantalla.	Renderizado reactivo desde ViewModels a las Vistas de Flutter.

2.2 Conceptos Clave

2.2.1 Overbooking (Sobreventa)

El *overbooking* ocurre cuando se reservan más habitaciones de las que físicamente están disponibles para una misma fecha. En HostSigchos, este problema se mitiga cruzando matemáticamente las fechas de *Check-in* y *Check-out* y bloqueando la reserva si el total ocupado iguala o supera el inventario disponible.

2.2.2 Clean Architecture y MVVM

Arquitectura Limpia es una filosofía de diseño donde las reglas del negocio (Domain) no dependen de bases de datos ni de interfaces (Data y Presentation). MVVM (*Model-View-ViewModel*) facilita esto al usar el ViewModel como intermediario para manejar el estado de la vista.

2.2.3 Transacciones Atómicas

Una transacción atómica en Firebase garantiza que un conjunto de lecturas y escrituras se completen de manera íntegra, o fallen por completo. HostSigchos usa esto para asegurar que si dos usuarios confirman el pago al mismo tiempo, solo uno obtenga la última habitación.

2.2.4 Geolocalización Operativa y Magnetometría

Uso del hardware GPS (Sistema de Posicionamiento Global) del dispositivo combinado con el magnetómetro (brújula) para posicionar al turista en el cantón Sigchos. Esto permite mostrar su orientación física real mediante cálculos de azimuth y trazar puntos cartográficos (hosterías) en el mapa interactivo.

2.2.5 Inteligencia Artificial Generativa y NLP

Los Modelos de Lenguaje Grande (LLMs), como Gemini de Google, procesan secuencias de texto mediante Redes Neuronales Transformer. En este proyecto se combinan con algoritmos de Procesamiento de Lenguaje Natural (NLP), Reconocimiento de Voz (*Speech-to-Text*) y síntesis vocal (*Text-to-Speech*) para crear un agente conversacional omnicanal y empático.

2.2.6 Sistemas Reactivos y WebSockets

La arquitectura orientada a eventos permite la comunicación bidireccional asíncrona. Mediante *WebSockets*, Firebase mantiene una conexión TCP persistente, logrando que cualquier

mutación de la base de datos (Ej. cuando el propietario aprueba una reserva) se notifique instantáneamente al cliente móvil sin necesidad de hacer *HTTP Long-Polling*.

2.2.7 Modelo de Seguridad Zero Trust y JWT

A diferencia de los modelos perimetrales clásicos, *Zero Trust* (Confianza Cero) asume que la red y el cliente ya están comprometidos. Las bases de datos backend exigen que cada solicitud esté firmada criptográficamente con un JSON Web Token (JWT) emitido por un proveedor OAuth 2.0. Las *Security Rules* validan esta firma matemática antes de autorizar cualquier mutación de datos.

2.2.8 Entornos de Ejecución Confiables (TEE)

La información biométrica nunca se transmite a la nube ni se expone al sistema operativo principal. Se cifra físicamente en un microprocesador aislado llamado *Secure Enclave* (iOS) o *Keystore* (Android). La validación se realiza a nivel de hardware local y solo devuelve un hash o token asíncrono a la aplicación.

2.2.9 Ofuscamiento y Mitigación de Ingeniería Inversa

La ingeniería inversa implica descompilar el archivo binario empaquetado (APK/AAB) para extraer propiedad intelectual, lógica de negocio o credenciales. El ofuscamiento es una técnica algorítmica de encriptación léxica que destruye y reduce los nombres de variables, clases y flujos (*symbols*) durante la compilación AOT (*Ahead-of-Time*), blindando el código fuente.

3 Arquitectura y Estructura

3.1 Arquitectura del Sistema

El desarrollo del aplicativo móvil en Flutter se sustenta fuertemente en los lineamientos de **Clean Architecture** definidos por Robert C. Martin [2], complementado con el patrón arquitectónico de interfaz de usuario **MVVM (Model-View-ViewModel)** mediado por la librería **Provider** [3].

3.1.1 Capas de Arquitectura Limpia

La principal meta de esta arquitectura es la mantenibilidad a largo plazo y la capacidad de realizar pruebas aisladas. Se divide estrictamente en tres capas independientes:

- **Capa Domain (Dominio - Lógica central):** Totalmente independiente de Flutter, Firebase o cualquier API externa. Define las **Entities** puras (como **Usuario**, **Reserva**, **Habitacion**), los **UseCases** (orquestadores de lógica de negocio, por ejemplo **VerificarDisponibilidadUseCase**) y los contratos abstractos de los repositorios.
- **Capa Data (Datos):** Su responsabilidad es consumir las interfaces definidas en el dominio. En ella residen los **DataSources** (por ejemplo **auth_datasource.dart** que consume la SDK de Firebase) y los **Models** (encargados de serializar y deserializar JSON **fromFirestore**, **toJson**).
- **Capa Presentation (Presentación - UI):** Implementa el patrón **MVVM**. Consta de los **ViewModels** que manejan variables reactivas y exponen métodos a las **Views** (Widgets de Flutter). Las vistas reaccionan al estado reconstruyéndose de forma limpia a través de `context.watch<T>()`.

3.1.2 Inversión de Dependencias (GetIt)

HostSigchos centraliza la inyección de todas las dependencias a través del archivo `injection\_container.dart` utilizando **GetIt**. Todas las clases, incluyendo los **ViewModels**, **UseCases** y **DataSources** se registran aquí. Esto desacopla totalmente a la interfaz de usuario de las implementaciones concretas, haciendo posible, por ejemplo, inyectar **Mock Repositories** durante la ejecución de los tests.

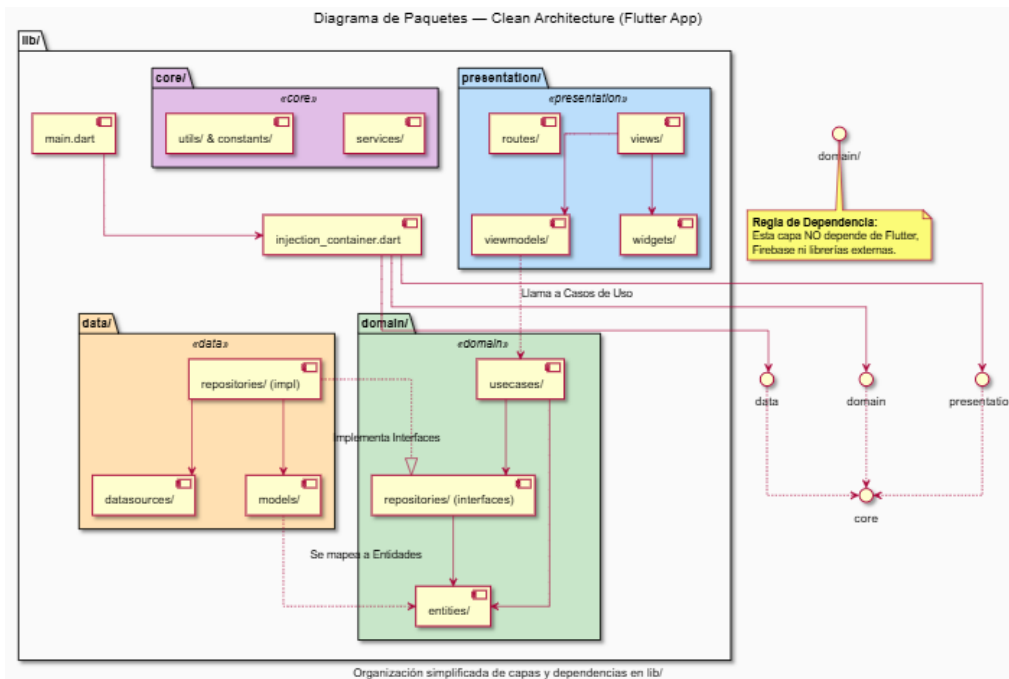


Figura 3.1: Flujo de datos bidireccional desde la capa de Presentación hasta el Origen de Datos mediado por los Casos de Uso.

3.1.3 Diagrama de Despliegue

Para comprender cómo interactúan físicamente los componentes de software desarrollados, se modeló la distribución de los nodos del sistema. Este diagrama evidencia la separación entre el dispositivo móvil (Turista), el navegador web (Propietario) y la infraestructura en la nube (Firebase).

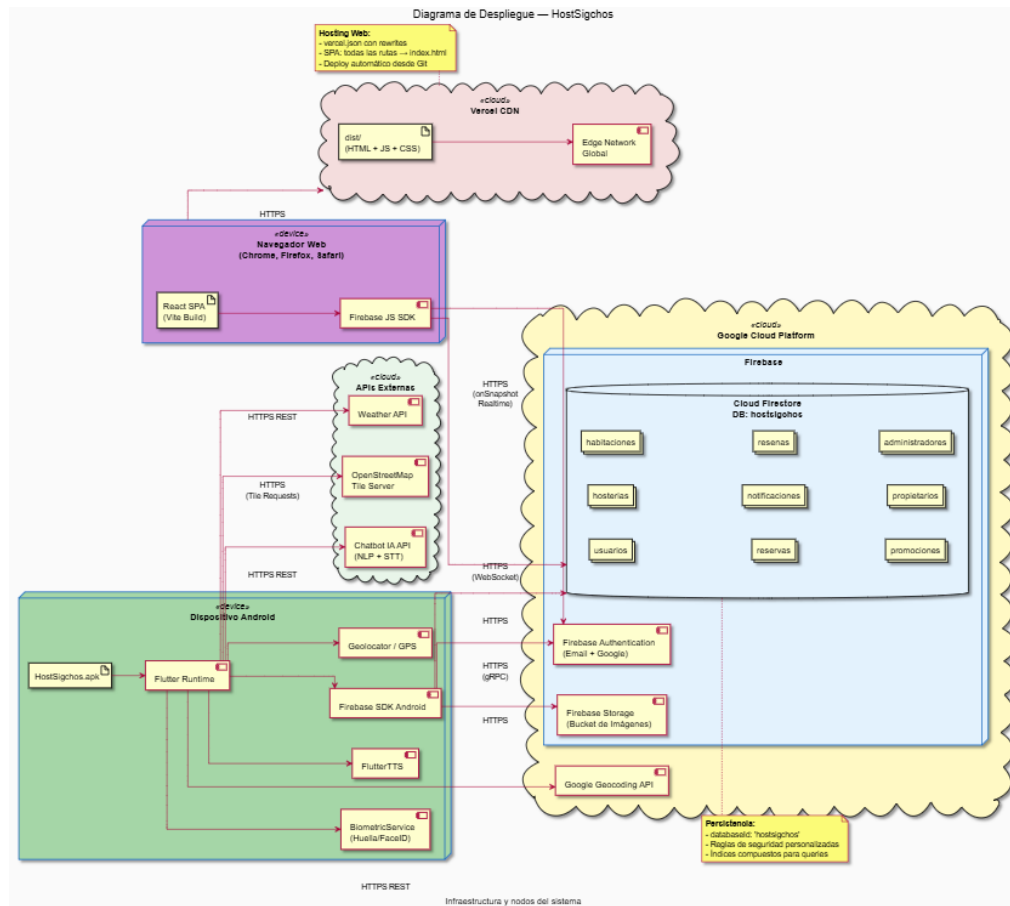


Figura 3.2: Diagrama de Despliegue mostrando la topología física y los protocolos de comunicación.

3.2 Stack Tecnológico, Herramientas y APIs Integradas

Para construir un ecosistema altamente escalable e interactivo, HostSigchos se apoyó en un robusto abanico de tecnologías, librerías y APIs de terceros:

- **Frameworks y Lenguajes:** **Flutter** con el lenguaje **Dart** para la aplicación móvil multiplataforma, y **React JS** estilizado con **Tailwind CSS** para el panel administrativo web.
- **Backend-as-a-Service (BaaS):** La infraestructura entera descansa sobre **Google Firebase**. Se utilizan los módulos de **Firebase Authentication** (OAuth 2.0 y Correo), **Cloud Firestore** (Base de datos NoSQL en tiempo real mediante WebSockets), **Firebase Storage** (Almacenamiento de imágenes de hosterías y perfiles) y **Firebase Hosting** (Para desplegar los documentos legales estáticos).
- **APIs de Inteligencia Artificial y Datos:** Integración asíncrona (RESTful) con la **Gemini API** (Google) para dotar de razonamiento al chatbot turístico, y **OpenWeather-Map API** para consultas climatológicas en tiempo real de la zona de Sigchos.

■ Librerías de Hardware Nativo:

- **local_auth**: Para vincular el *Secure Enclave* y habilitar autenticación con huella dactilar o FaceID.
- **geolocator** y **flutter_compass**: Para acceder a la antena GPS y al magnetómetro (brújula) del celular.
- **flutter_tts** y **speech_to_text**: Para proveer síntesis y reconocimiento de voz bidireccional en el chatbot.

■ Servicios Cartográficos: Se integró el motor **flutter_map** (basado en *Leaflet*) consumiendo la capa de mapas de código abierto **OpenStreetMap**.

■ Herramientas de Arquitectura: **Provider** como gestor de estado reactivo, **GetIt** para la inyección de dependencias, y el paquete **shimmer** para la ergonomía visual durante tiempos de carga por red (*latencia*).

■ Asistencia de Desarrollo (Ingeniería de Software): Se empleó la plataforma **Google Antigraity** (IA Agentiva) para estandarizar la base de código y asegurar el cumplimiento de principios SOLID durante la estructuración de la Arquitectura Limpia, y el empaquetador **MiKTeX** para compilar el presente documento LaTeX.

3.3 Diagrama de carpetas

La estructura física del proyecto móvil es un reflejo de la arquitectura descrita. La raíz ‘lib/’ está compuesta por:

```
HostSigchos/lib/
|-- core/                # Elementos transversales al proyecto
|   |-- constants/       # Rutas de Firestore, variables ENV
|   |-- errors/          # Manejo global de excepciones (Failures)
|   |-- l10n/            # Archivos de internacionalización
|   |-- services/        # Servicios de Hardware (Biometric, Notification)
|   +-- utils/           # Validadores, formateadores de fechas/precios
|
|-- data/                # Capa de Acceso a Datos
|   |-- datasources/
|   |   |-- api/         # (Ej. openweathermap api)
|   |   |-- firebase/    # (Ej. auth_datasource.dart, reserva_datasource.dart)
|   |   +-- remote/      # (Ej. geocoding y chatbot Gemini api)
|   |-- models/          # Clases con fromJson/toJson
|   +-- repositories/    # Implementaciones concretas (...RepositoryImpl.dart)
|
|-- domain/              # Capa de Dominio Pura (Dart Vainilla)
|   |-- entities/        # Entidades como Reserva, Habitación, Hosteria
|   |-- repositories/    # Contratos/Interfaces abstractas
|   +-- usecases/        # Logica de negocio orquestada
|       |-- auth/
|       |-- habitacion/   # Ej: check_disponibilidad_usecase.dart
|       +-- reserva/     # Ej: crear_reserva_usecase.dart
```

```
|  
|-- presentation/           # Capa Visual y Gestores de Estado  
|   |-- routes/             # Tabla de enrutamiento (app_routes.dart)  
|   |-- viewmodels/         # Gestores de estado (AuthViewModel, ReservaViewModel)  
|   |-- views/              # Pantallas UI (19 pantallas en total)  
|       |-- auth/  
|       |-- chatbot/  
|       |-- hosteria/  
|       +-- reserva/  
| +-- widgets/              # Componentes UI reusables (Botones, Modales, Tarjetas)  
|  
|-- themes/                 # Definicion del ThemeData general (Colores, Tipografias)  
|-- injection_container.dart # Locator (GetIt) para inyeccion de dependencias  
+-- main.dart               # Inicializacion de Firebase, GetIt, Providers y RunApp
```

3.4 Modelo de datos

El backend principal se sostiene en **Cloud Firestore**. A pesar de ser una base de datos NoSQL documental, se modeló cuidadosamente simulando claves foráneas (como `hosteriaId` o `usuarioId`) que permiten realizar consultas robustas.

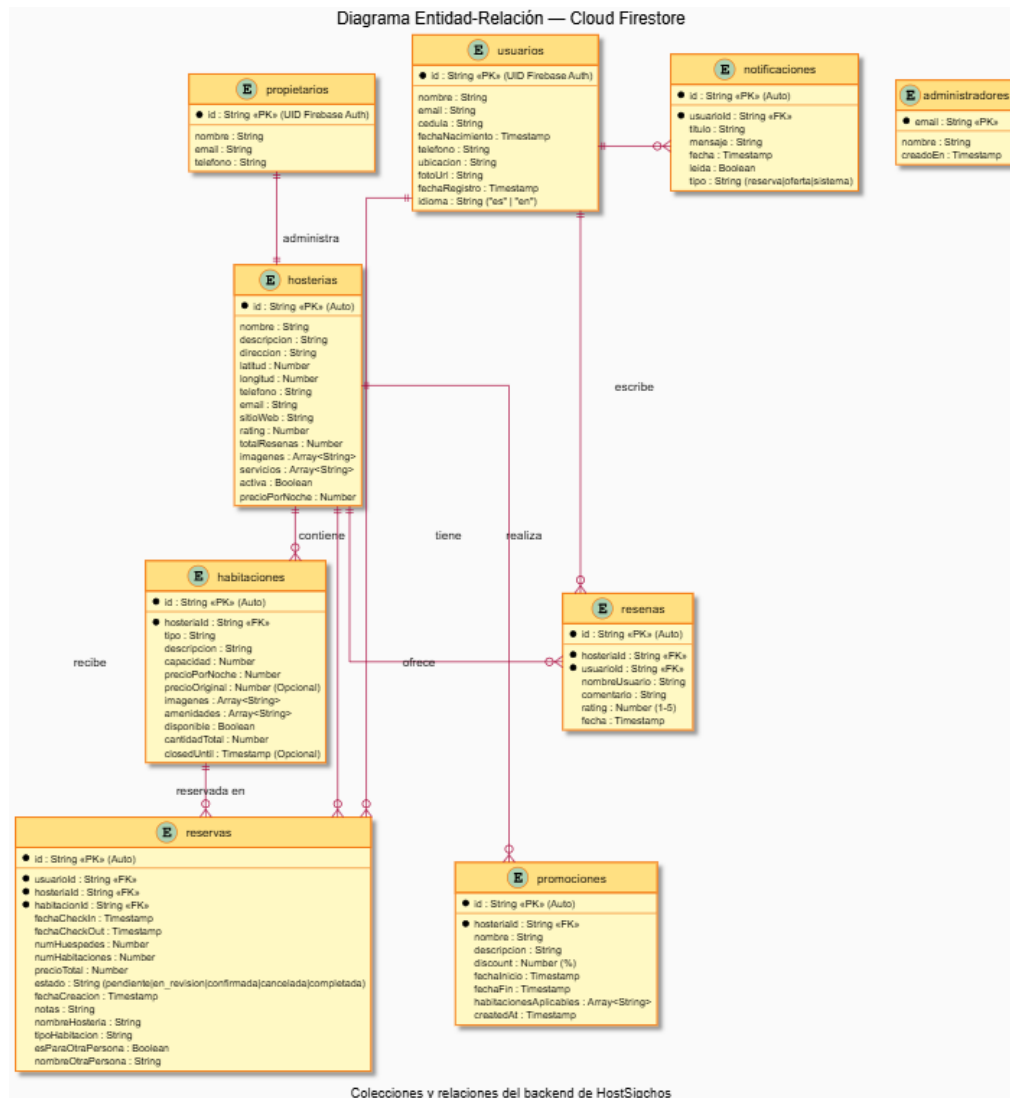


Figura 3.3: Estructura documental y referencias simuladas (Foreign Keys) en Cloud Firestore.

Las colecciones principales operativas son:

- **usuarios**: Almacena información del turista (cédula, correo, avatar vinculado a Firebase Storage) y el UID proporcionado por Firebase Auth.
- **hosterías**: Almacena coordenadas geográficas, imágenes de carrusel, amenidades, calificación media (rating) y descripción.
- **habitaciones**: Colección dependiente lógicamente de la hostería (relacionada vía `hosteriaId`). Contiene precio por noche, cantidad disponible, capacidad y tipo de cama.
- **reservas**: Documento transaccional central que guarda los detalles temporales (Check-in, Check-out), montos financieros (`precioTotal`) y el estado (Pendiente, Confirmada, Cancelada, Completada).

3.5 Diagramas UML del Sistema

Para proporcionar una vista integral de la ingeniería de software, se extrajeron los siguientes modelos unificados del proyecto, que abarcan la estructura estática, el comportamiento dinámico y la arquitectura:

3.5.1 Diagrama de Componentes (Clean Architecture)

Muestra la división macroscópica de los grandes bloques de software en capas de Dominio, Datos y Presentación, demostrando la alta cohesión y el bajo acoplamiento del código fuente.

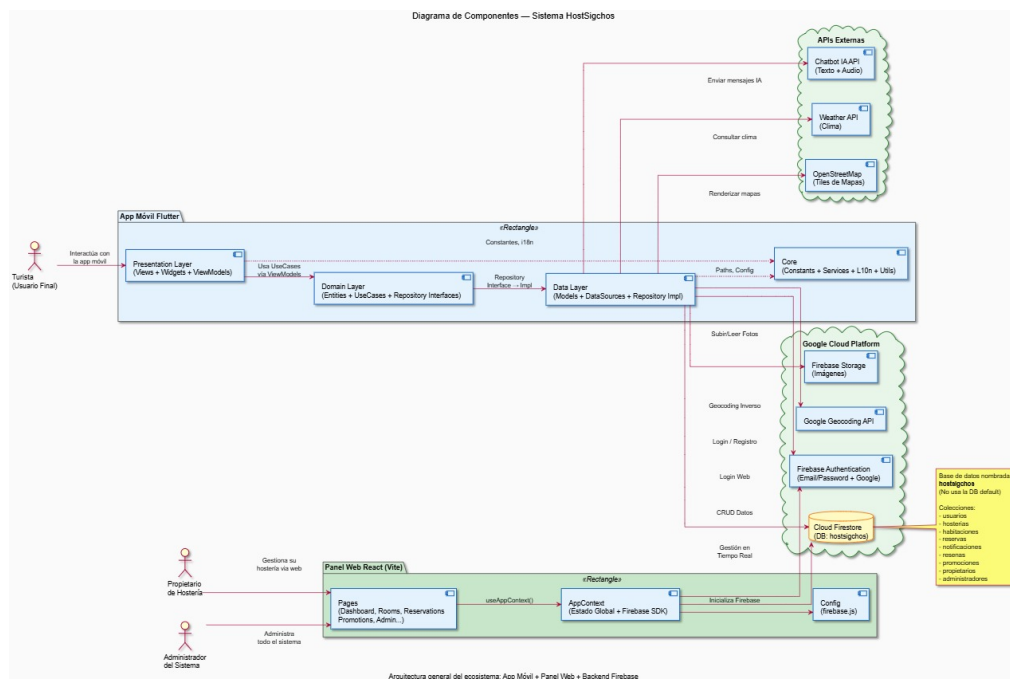


Figura 3.4: Diagrama de Componentes de la Arquitectura.

3.5.2 Diagrama de Clases del Dominio

Este diagrama ilustra la estructura estática de las Entidades Vainilla (Usuario, Reserva, Hostería, Habitación) y las relaciones semánticas entre ellas, siendo el núcleo de la aplicación, libre de dependencias de bases de datos externas.

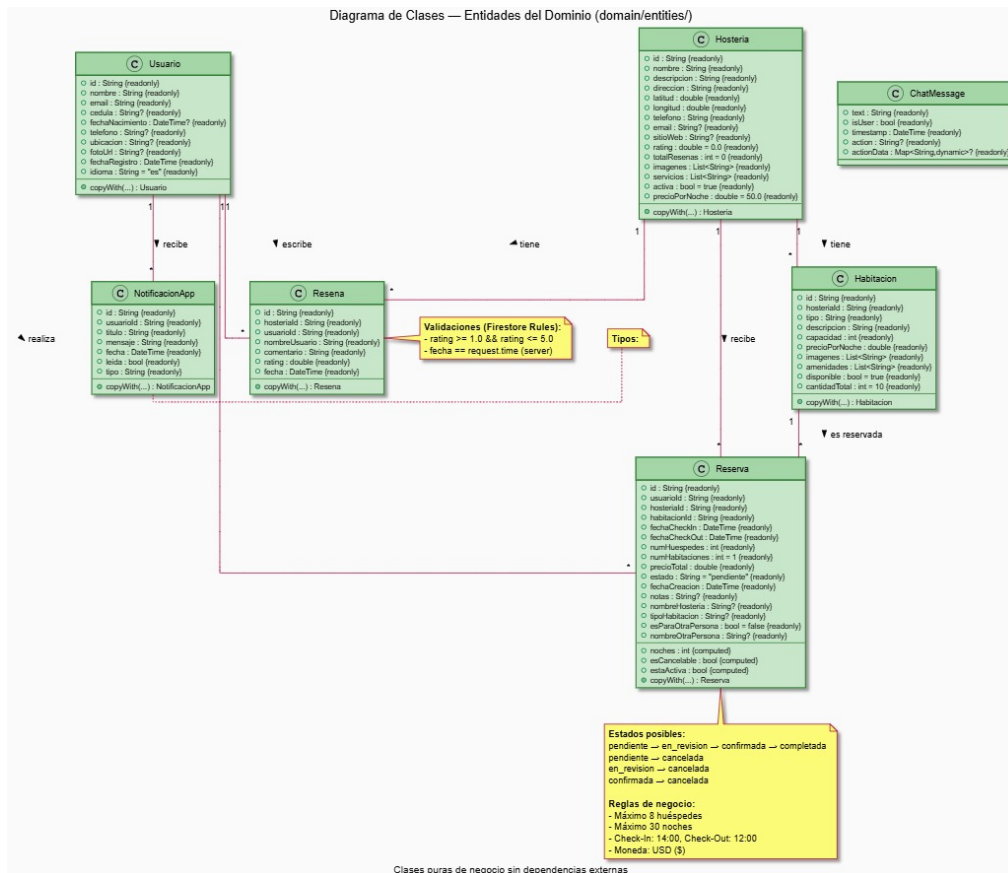


Figura 3.5: Diagrama de Clases de las Entidades de Dominio.

3.5.3 Diagrama de Casos de Uso

Muestra la interacción fundamental entre los Turistas (Usuarios Móviles) y los Propietarios (Administradores) con los módulos principales del ecosistema, como exploración de hosterías, uso de IA, reservas y gestión de inventario.

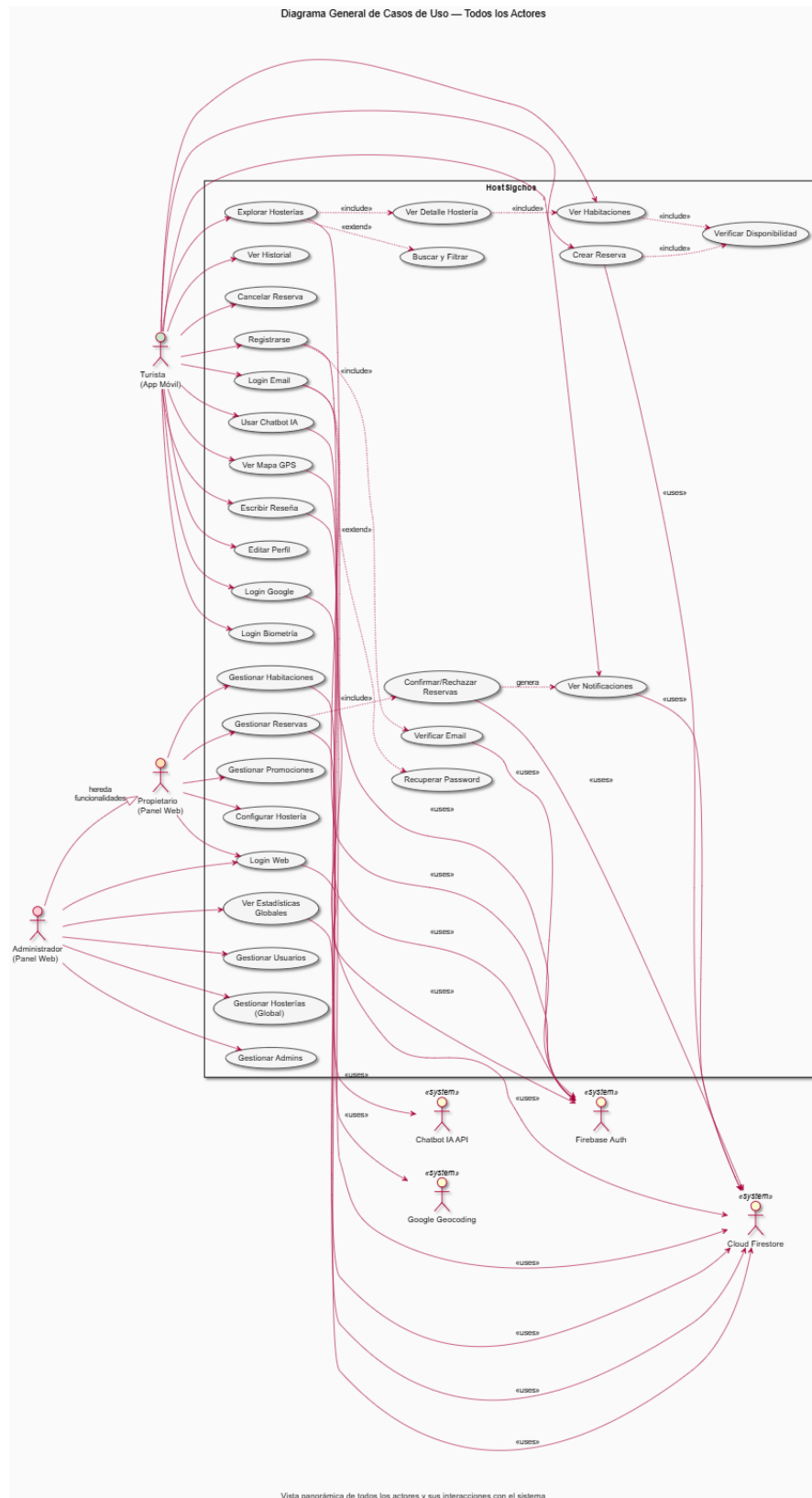


Figura 3.6: Diagrama de Casos de Uso general del sistema HostSigchos.

3.5.4 Diagrama de Secuencia (o Actividad)

Detalla el flujo lógico de comunicación transaccional entre el Cliente (Móvil), el Controlador de lógica (UseCase) y la Base de Datos (Firestore) para prevenir el cruce de fechas.

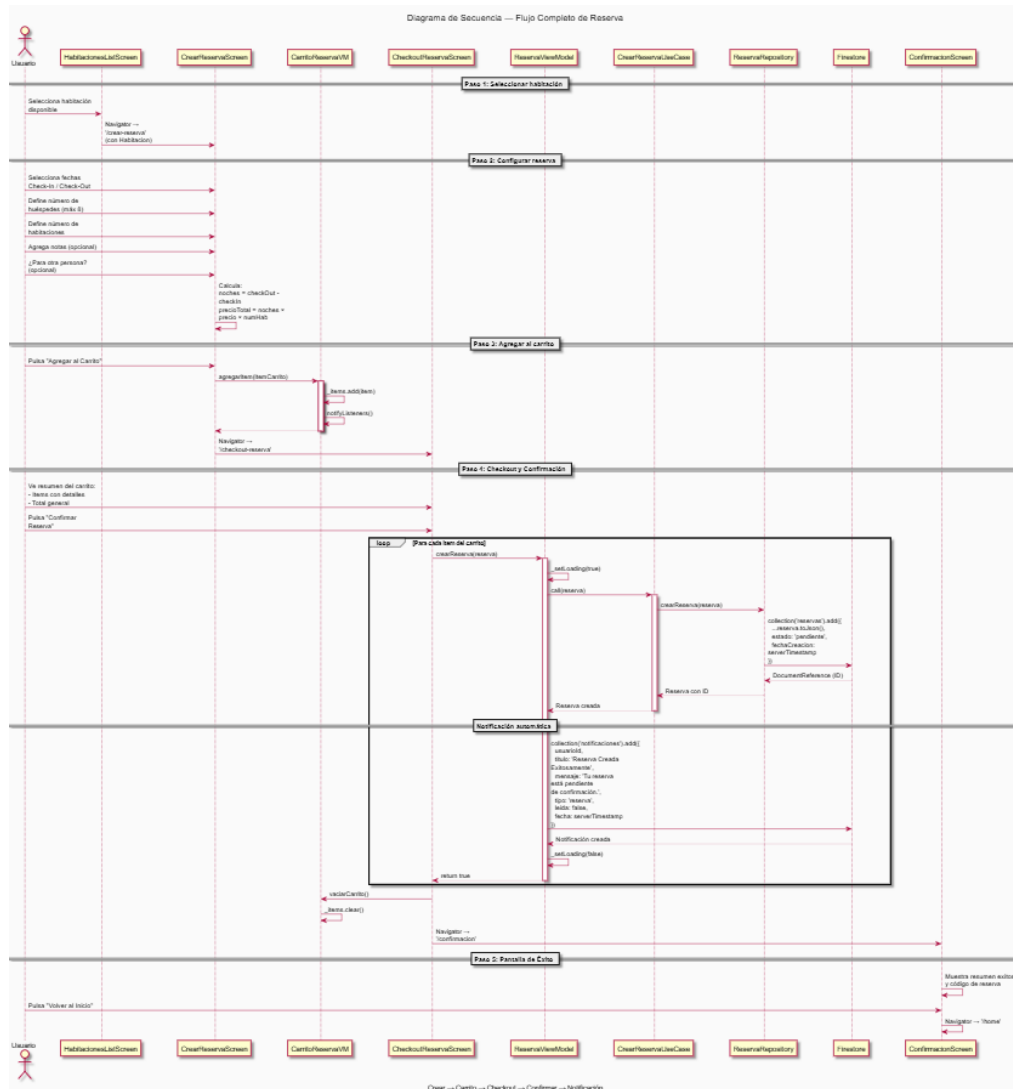


Figura 3.7: Diagrama UML del flujo de reservas y validación.

3.5.5 Máquina de Estados de Reserva

Demuestra el ciclo de vida estricto de un documento de reserva dentro del ecosistema, mutando su estado asíncronamente (Pendiente, Confirmada, Cancelada, Completada) según la interacción del Administrador.

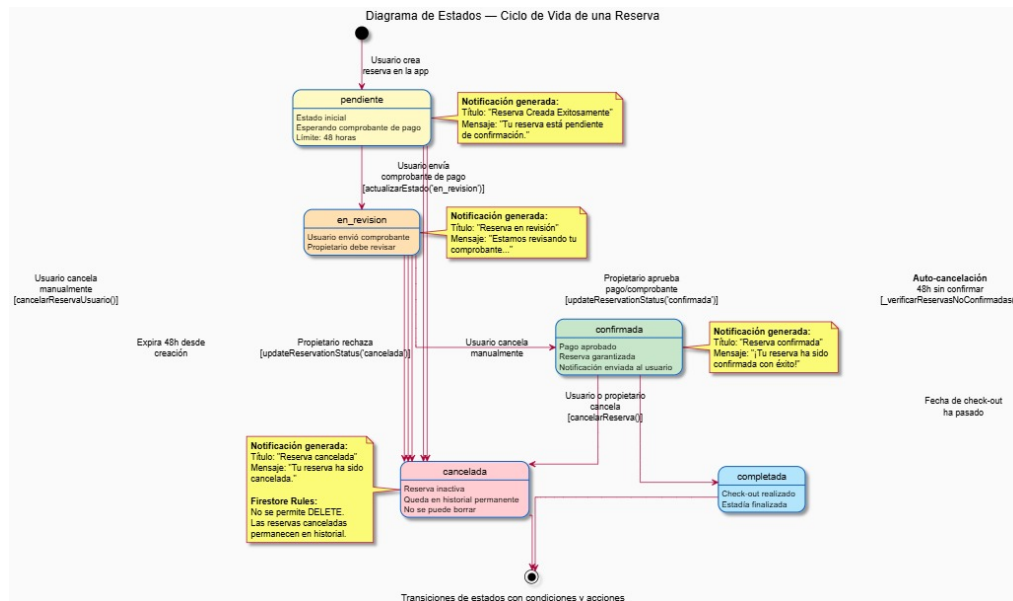


Figura 3.8: Diagrama de Estados para el ciclo de vida transaccional.

3.5.6 Diagrama Entidad-Relación Extendido (ER)

A pesar de que Firebase Firestore es una base de datos documental (NoSQL), HostSigchos maneja dependencias lógicas fuertes. Este diagrama refleja cómo se conectan los documentos mediante referencias (Ej: `hosteriaId`), simulando claves foráneas convencionales.

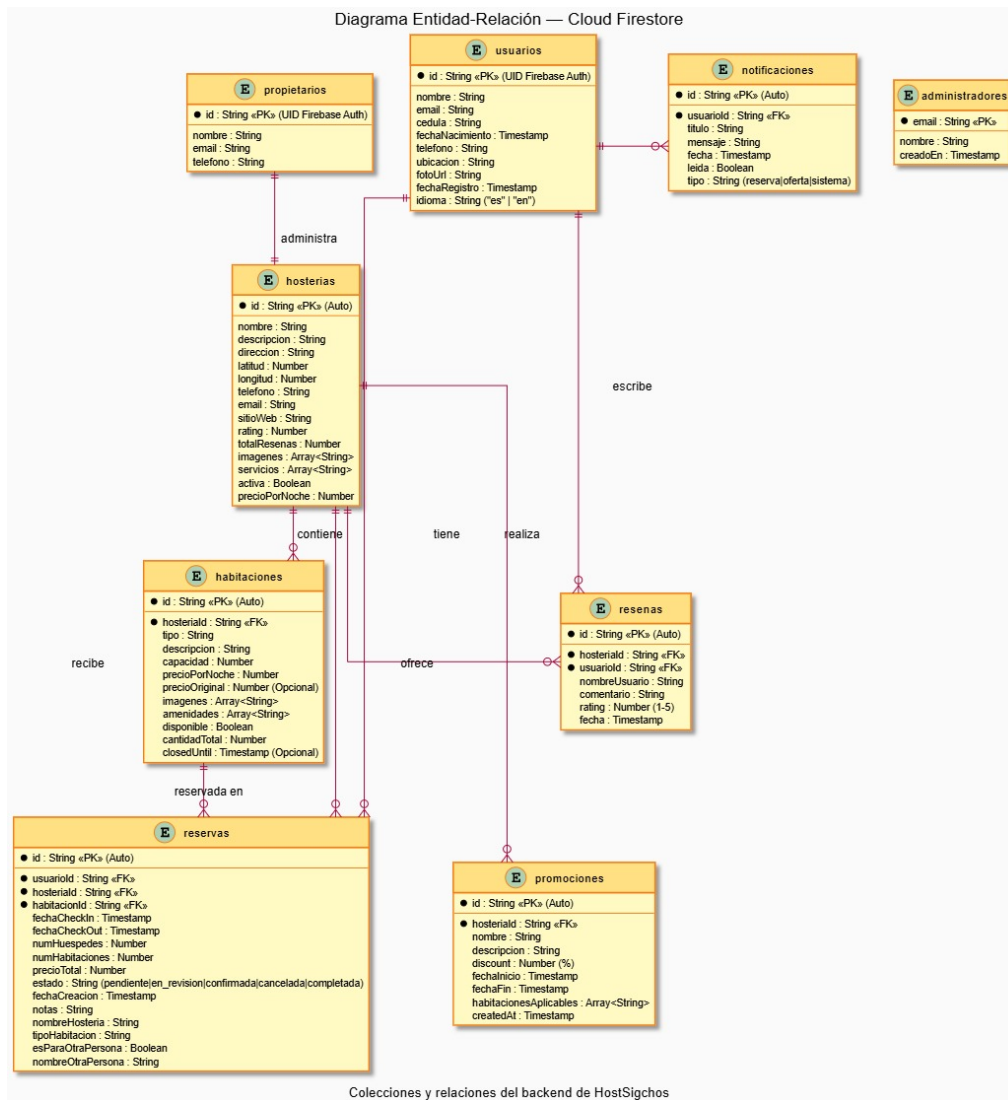


Figura 3.9: Diagrama ER lógico de las colecciones de Firestore.

4 Desarrollo y capturas del aplicativo

El sistema abarca diversas funcionalidades complejas que requieren sincronización asíncrona y uso intensivo del hardware del dispositivo móvil. Durante el ciclo de vida del proyecto, se estandarizó la base de código empleando arquitecturas limpias asistidas por **Antigravity**, una IA agentiva que aseguró el cumplimiento de principios SOLID. Se desarrollaron en total 19 rutas de pantalla (incluyendo interfaces móviles y web) completamente funcionales y validadas, orquestadas mediante 11 ViewModels (Provider) y 7 entidades de negocio. Adicionalmente, se integró una política incipiente de caché offline para mitigar pérdidas de conexión en zonas geográficas apartadas. A continuación se detalla con precisión milimétrica el desarrollo de todos los módulos.

4.1 Módulo Web y Entorno Legal

Previo al uso de la plataforma, los usuarios y propietarios interactúan con el portal web institucional. Este módulo aloja la **Landing Page** promocional, la cual explica las virtudes del sistema y fomenta la descarga del aplicativo móvil. Además, desde esta pantalla inicial, el usuario tiene la capacidad de cambiar el idioma de toda la plataforma gracias a un selector de internacionalización (i18n), garantizando accesibilidad inmediata para turistas extranjeros. Asimismo, debido a las normativas estrictas de las tiendas de aplicaciones (Play Store y App Store), se redactaron e inyectaron las normativas legales mediante páginas web estáticas alojadas en Firebase Hosting.

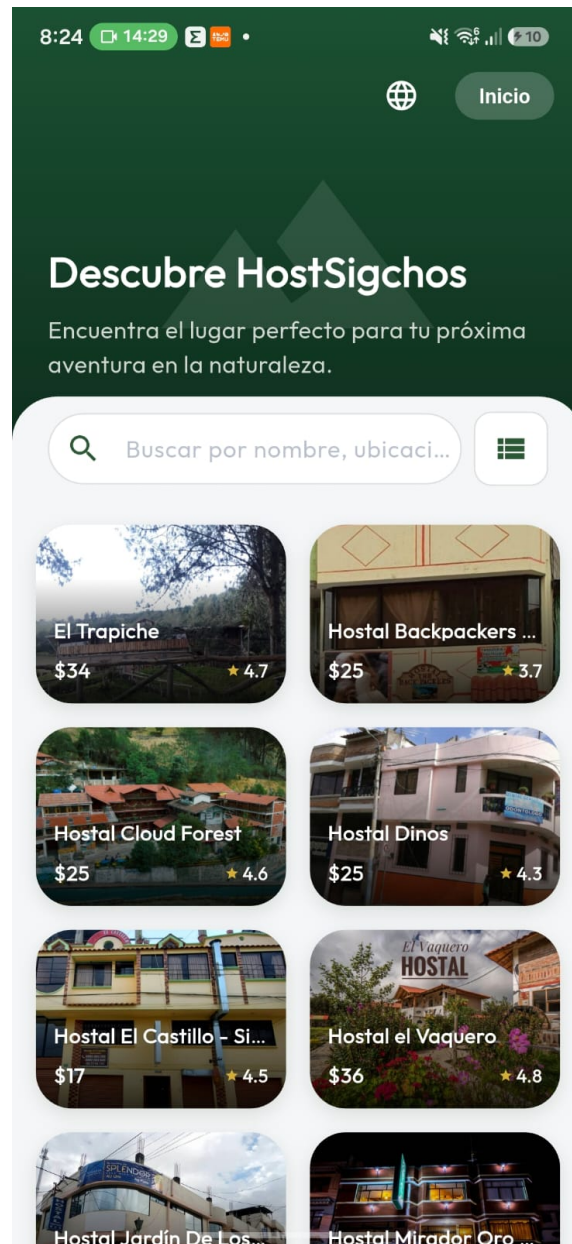


Figura 4.1: Landing Page promocional del ecosistema HostSigchos.

El usuario puede revisar las políticas y términos en cualquier momento, tanto desde la web como desde la aplicación móvil.

- **Políticas de Privacidad:** <https://hostsigchos.web.app/politicas.html>
- **Términos y Condiciones:** <https://hostsigchos.web.app/terminos.html>

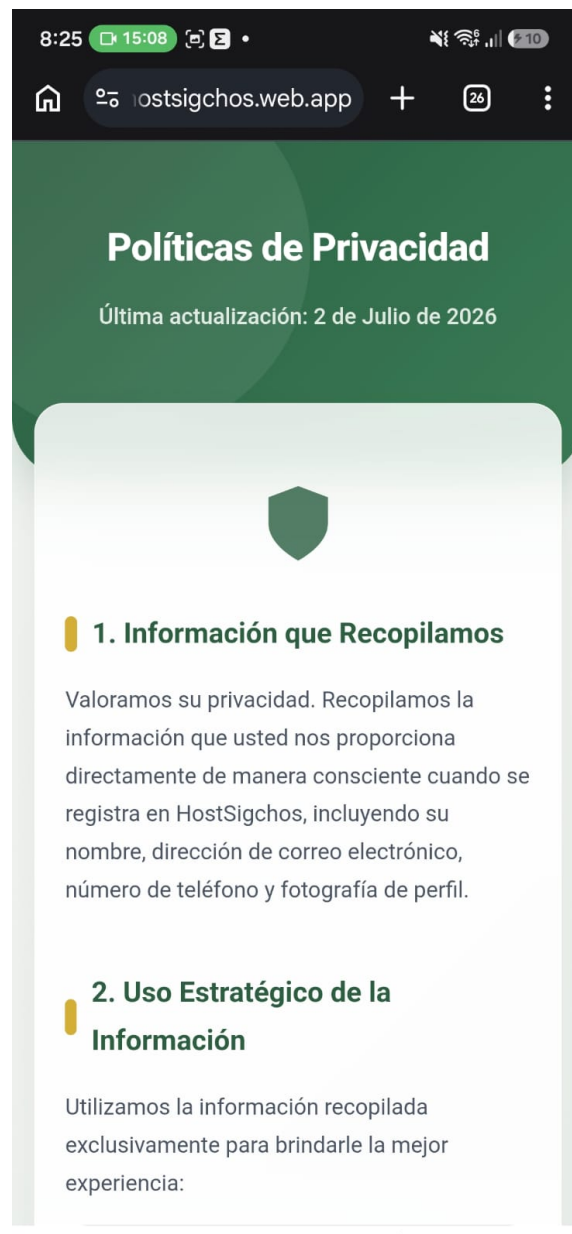


Figura 4.2: Visualización de las Políticas de Privacidad.

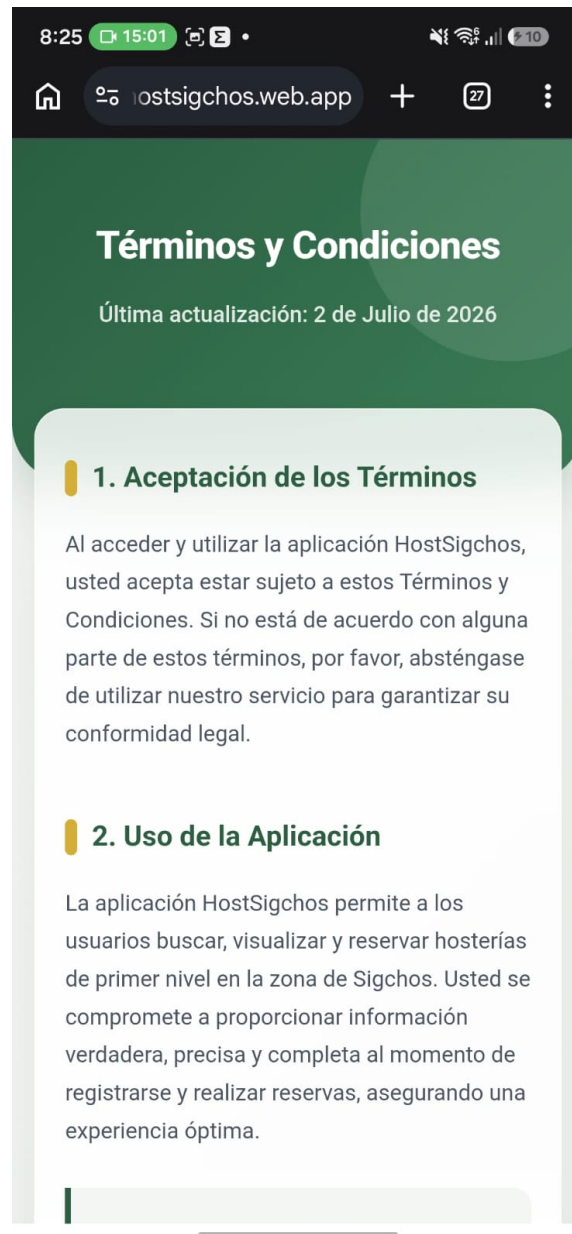


Figura 4.3: Documento legal de Términos y Condiciones.

4.2 Módulo de Autenticación, Registro y Biometría

El sistema de seguridad está diseñado para proteger los datos del turista y facilitar el acceso. El flujo principal comienza con la creación de una cuenta en Firebase Auth. El usuario puede ingresar mediante correo electrónico y contraseña o mediante un proveedor de OAuth 2.0 (Google Sign-In). Desde el punto de vista del desarrollo, el `AuthViewModel` maneja los estados de *cargando* (mostrando indicadores de progreso circular) y captura excepciones como contraseñas débiles o correos ya registrados, devolviendo al usuario un *SnackBar* amigable.



Figura 4.4: Pantalla de inicio de sesión clásica y SSO (Google).



The screenshot shows a mobile application interface for registration. At the top, the status bar displays the time 12:00, signal strength, and battery level at 57%. The app header features a back arrow and the title "Registrarse". Below the header is a profile picture placeholder with a camera icon. The form consists of several input fields: "Nombre completo" (with a person icon), "Correo electrónico" (with an envelope icon), "Cédula" (with a dropdown arrow and a document icon), "Fecha de nacimiento" (with a calendar icon), a phone number field with a "+593" country code selector and a telephone icon, "País de origen" (with a flag icon and a dropdown arrow), "Ciudad / Provincia" (with a building icon and a dropdown arrow), and "Contraseña" (with a lock icon and a toggle visibility icon). The form is styled with rounded corners and a light gray background.

Figura 4.5: Pantalla de registro con validaciones de formulario en tiempo real.

En caso de que el usuario pierda su acceso, se implementó un flujo de **Recuperación de Contraseña** mediante un enlace seguro enviado al correo del turista, posibilitando el auto-servicio en la gestión de credenciales.



Figura 4.6: Módulo de recuperación de contraseña vía correo electrónico.

Adicionalmente, desde la propia pantalla de Login, el usuario puede acceder al selector de idioma para adaptar la interfaz a su lengua materna de manera instantánea, sin necesidad de autenticarse previamente.

Para elevar el estándar de seguridad y mejorar la experiencia de usuario (UX), se integró la verificación biométrica del hardware (Huella Dactilar o Reconocimiento Facial) usando la librería **local_auth**. Esta validación se apoya en el *Secure Enclave* del dispositivo para evitar robos de identidad [4].

```
// Función inyectada en el AuthViewModel
Future<bool> autenticarConBiometria() async {
  final LocalAuthentication auth = LocalAuthentication();
```

```
bool canAuthenticateWithBiometrics = await auth.canCheckBiometrics;

if (canAuthenticateWithBiometrics) {
  return await auth.authenticate(
    localizedReason: 'Por favor, autentícate para acceder a HostSigchos',
    options: const AuthenticationOptions(biometricOnly: true),
  );
}
return false; // Fallback a contraseña manual
}
```

Listing 4.1: Lógica asíncrona de autenticación biométrica utilizando el hardware nativo.

Figura 4.7: Autenticación biométrica habilitada por el hardware nativo.

4.3 Módulo de Exploración (Dashboard) y Visualización de Hosterías

Una vez autenticado, el turista ingresa al **Dashboard** principal. Esta pantalla consume el **HosteriaViewModel**, recuperando la lista de hosterías desde **Firestore**. Para contrarrestar problemas de red (alta latencia) y evitar bloqueos en la UI, se implementó el paquete **shimmer**, el cual renderiza un esqueleto gris animado mientras los datos terminan de descargar. El usuario tiene la opción de alternar la vista entre un layout de **Lista** y uno de **Cuadrícula**, maximizando la ergonomía visual. Las imágenes pesadas se cachean asíncronamente vía **cached_network_image**.

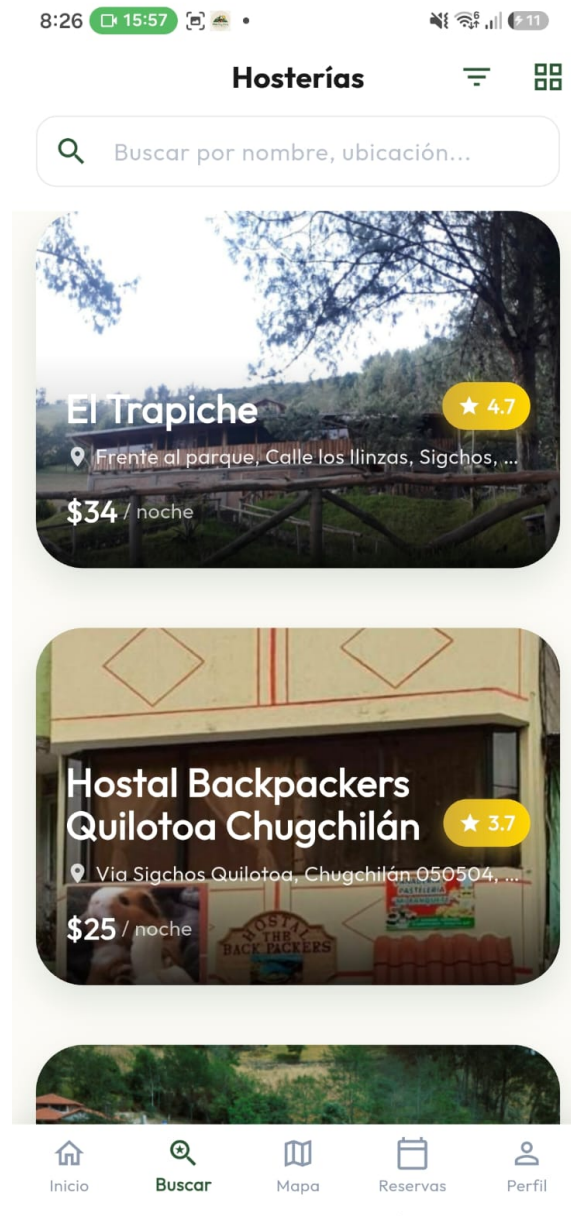


Figura 4.8: Dashboard principal de exploración en formato Lista.

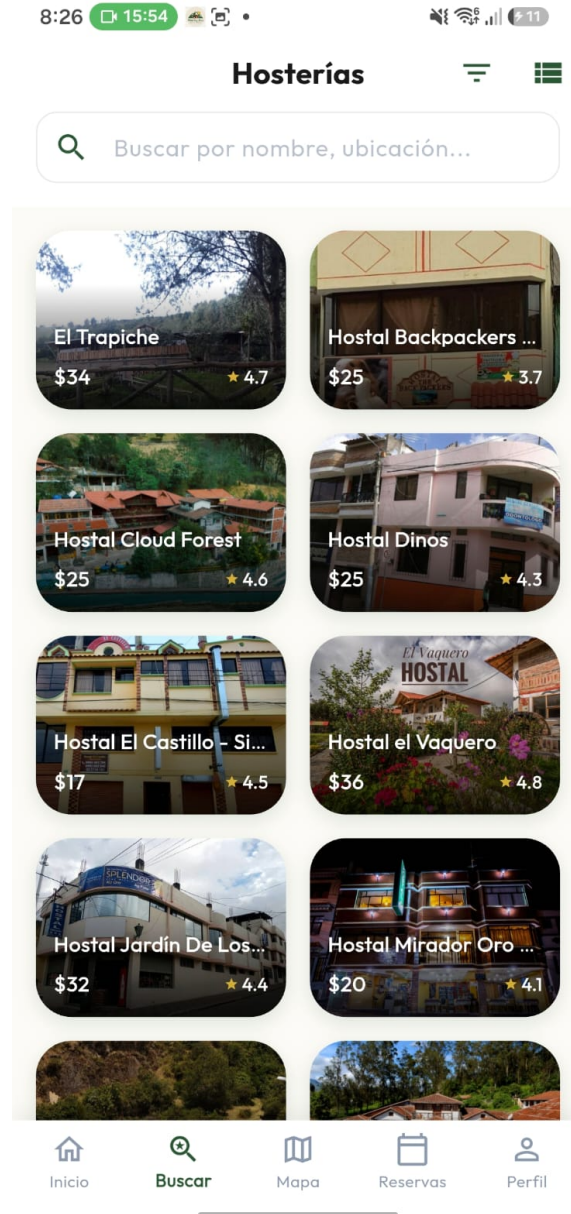


Figura 4.9: Dashboard principal de exploración alternado al formato Cuadrícula.

Al seleccionar una hostería, se navega al detalle de la misma. Aquí el usuario puede visualizar un carrusel dinámico de las instalaciones, chips de amenities (Wi-Fi, piscina, etc.) y acceder al **Apartado de Reseñas** donde los huéspedes previos califican con estrellas su estadía, brindando retroalimentación comunitaria.



Figura 4.10: Vista detallada de los servicios generales de una hostería.

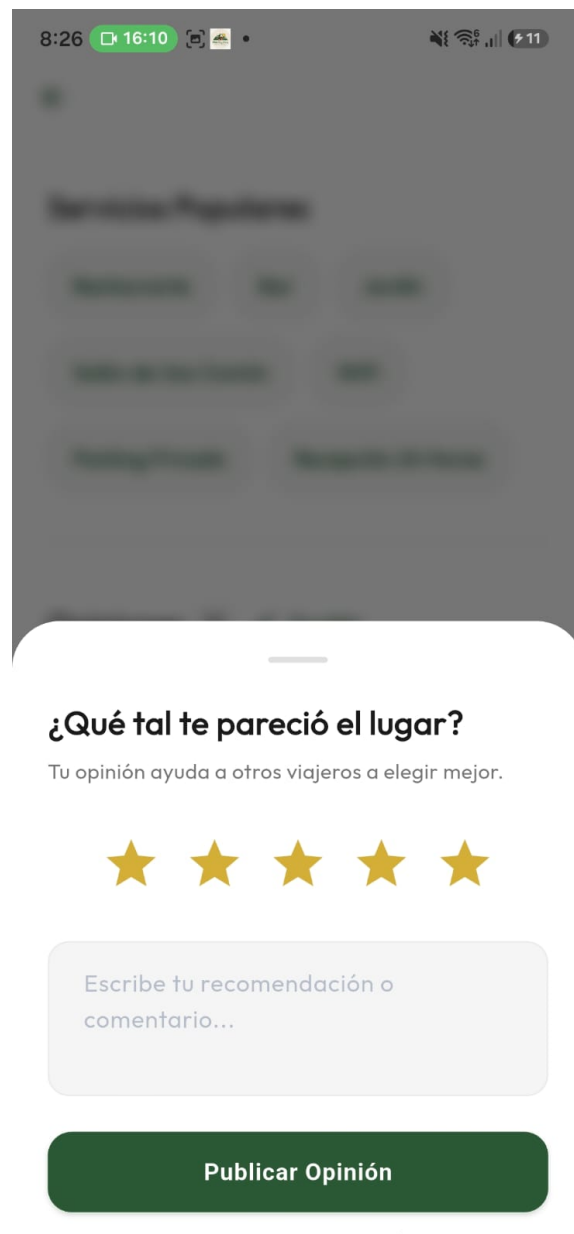


Figura 4.11: Módulo comunitario de calificación y reseñas.

Dentro del detalle, el turista puede acceder a la **Visualización de Habitaciones**, donde se despliegan las capacidades, tipos de cama y precios de cada alcoba disponible.

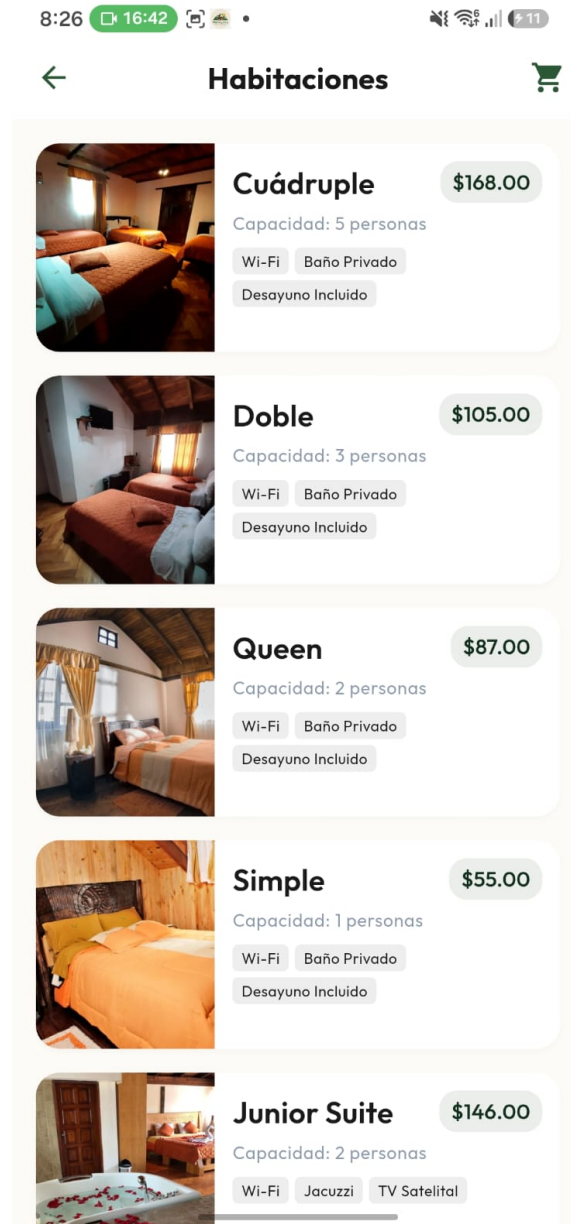


Figura 4.12: Catálogo detallado de habitaciones y precios por noche.

4.4 Módulo de Mapas Interactivos y Geolocalización

El turismo requiere orientación espacial precisa. Se incorporó un módulo de mapas implementado con el paquete **flutter_map** (motor Leaflet) y las capas base de OpenStreetMap. A nivel de desarrollo, se utiliza el paquete **geolocator** para solicitar permisos de GPS y obtener la latitud y longitud actual del usuario. Adicionalmente, se integró el paquete **flutter_compass** para hacer que el mapa rote según la orientación física del teléfono. Los *Markers* (puntos de interés) se renderizan leyendo las coordenadas de las hosterías alojadas en la base de datos, lo que permite trazar rutas y distancias estimadas.

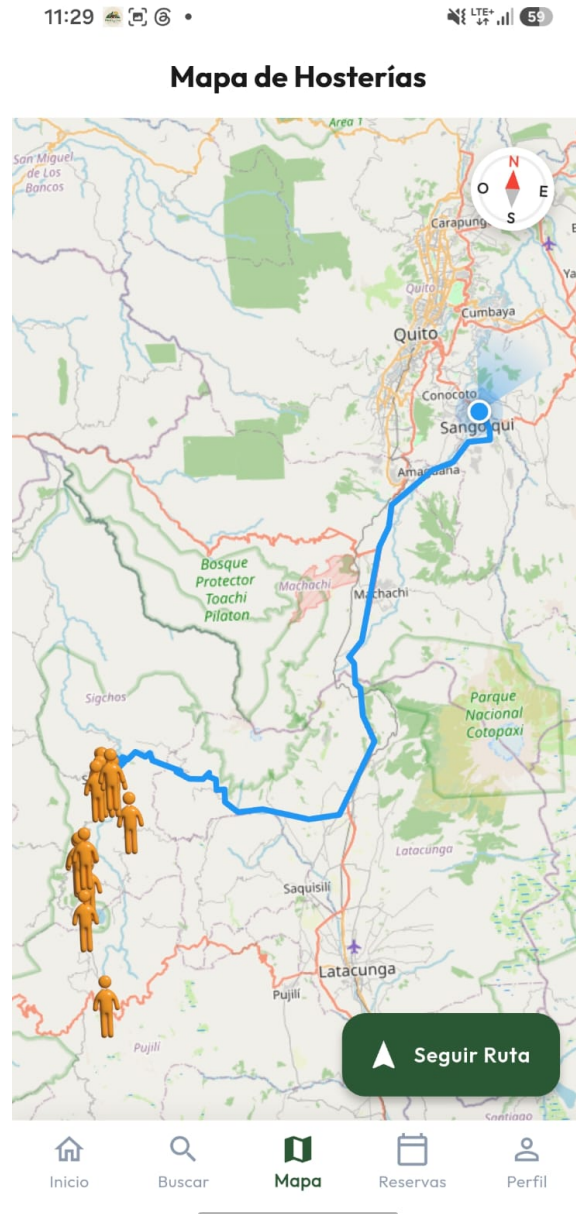


Figura 4.13: Pantalla de visualización de mapas integrando los marcadores de hosterías y geolocalización.

4.5 Asistente Virtual (Chatbot con Inteligencia Artificial)

Considerando la necesidad de guías turísticos 24/7, se construyó un Chatbot inteligente alimentado por la API de **Gemini** (Google). El `ChatbotViewModel` gestiona el historial de la conversación (prompts) en una lista local. El sistema inyecta un contexto inicial (*System Instruction*) indicándole a la IA que asuma el rol de guía experto en Sigchos.

Durante la fase de pruebas del sistema, se detectaron errores de saturación (Rate Limit 429) al

consumir la API gratuita de Gemini. Esto se solventó abstrayendo el manejo de error y devolviendo un estado controlado a la UI que invita al usuario a reintentar en lugar de colapsar la aplicación. Además, el módulo interactúa asincrónicamente con la API de **OpenWeatherMap** para brindar respuestas del clima en tiempo real, y utiliza los paquetes nativos de **flutter_tts** y **speech_to_text** para interacción por voz.

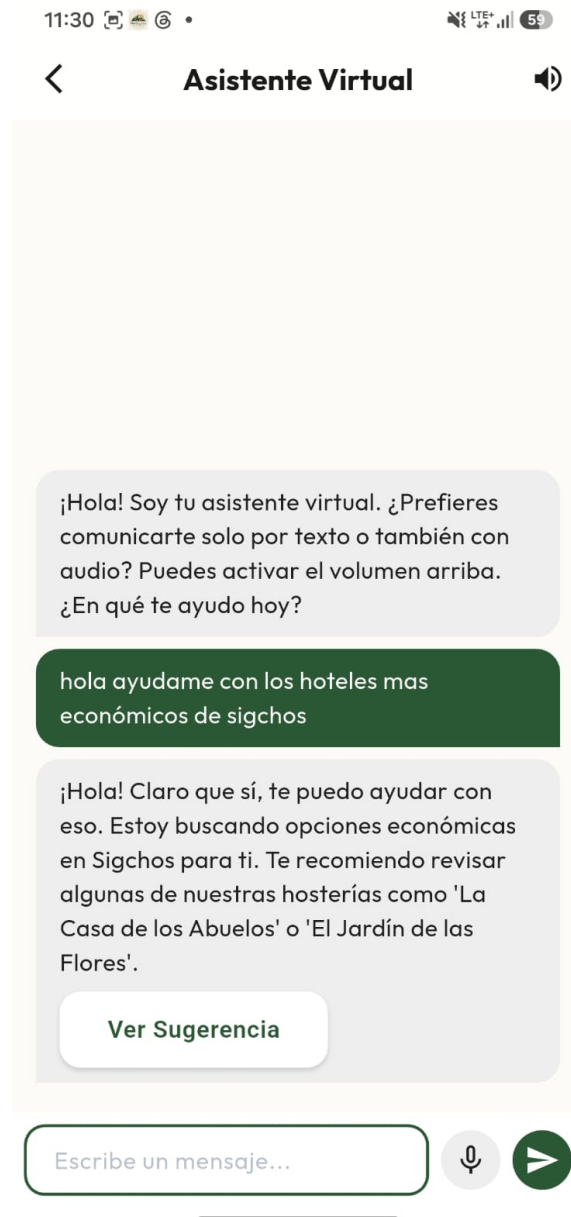


Figura 4.14: Asistente turístico integrado, respondiendo con texto y voz.

4.6 Proceso de Reserva y Resolución del Overbooking

El flujo transaccional es el núcleo de HostSigchos. Una vez que el usuario selecciona una habitación ingresa al **Módulo de Reservas**, donde debe introducir su fecha de *Check-in* y *Check-out* a través de un selector de calendario (`DateRangePickerDialog`).

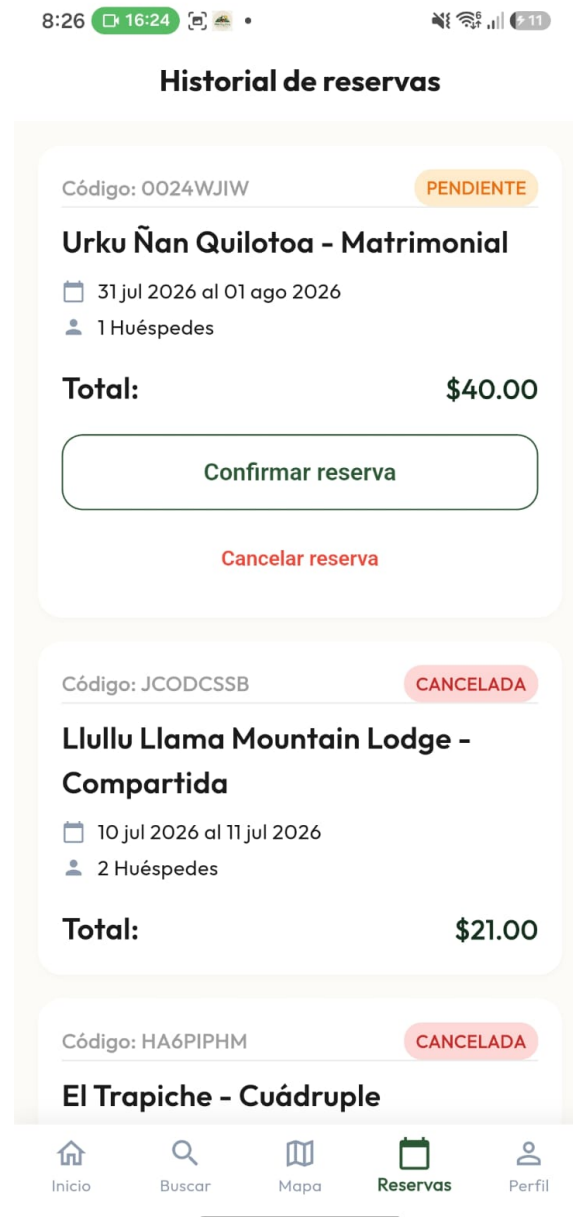


Figura 4.15: Módulo centralizado de gestión y solicitud de nueva reserva.

Para prevenir el *Overbooking* (sobreventa), el `VerificarDisponibilidadUseCase` recupera todas las reservas con estado “Pendiente” o “Confirmada” para dicha habitación y ejecuta un motor matemático de colisiones temporales en milisegundos:

```
// Dentro de domain/usecases/habitacion/verificar_disponibilidad_usecase.dart
bool isDisponible = true;
int habitacionesOcupadas = 0;

for (var r in reservasExistentes) {
  // Si la nueva fecha de ingreso es ANTES de la salida existente
  // Y la nueva fecha de salida es DESPUES del ingreso existente, hay cruce.
  if (checkIn.isBefore(r.fechaCheckOut) && checkOut.isAfter(r.fechaCheckIn)) {
    habitacionesOcupadas += r.numHabitaciones;
  }
}
if(habitacionesOcupadas >= habitacion.cantidadTotal) {
  isDisponible = false; // Se bloquea la reserva
}
```

Listing 4.2: Lógica pura en Dart para prevención de superposición temporal de fechas.



Figura 4.16: Selector de rango de fechas para validación de disponibilidad.



Figura 4.17: Proceso de *Checkout* detallando costos antes del guardado final.

4.7 Módulo de Usuario, Ajustes y Notificaciones

El perfil del turista en HostSigchos es altamente personalizable. El usuario puede acceder a la sección de **Editar Perfil** para actualizar sus datos de contacto, contraseña y fotografía, la cual es cargada directamente a los buckets seguros de **Firestore Storage**. Desde este mismo apartado, el usuario posee el control total para cambiar el **Idioma** de la aplicación, el cual es inyectado globalmente mediante un **LocaleProvider**.

```
// provider/locale_provider.dart
class LocaleProvider extends ChangeNotifier {
```

```
Locale _locale = const Locale('es'); // Español por defecto

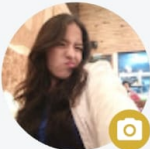
Locale get locale => _locale;

void setLocale(Locale newLocale) {
  if (!L10n.all.contains(newLocale)) return;
  _locale = newLocale;
  notifyListeners(); // Re-renderiza toda la App reactivamente
}
}
```

Listing 4.3: Lógica del LocaleProvider para el cambio de idioma en tiempo real.

8:27 17:11

< **Editar perfil**



Nombre completo

 Danna Andrade

Cédula

Céd  1722651567

Fecha de nacimiento

 15/06/2005

Teléfono

+593  0939185134

 Ecuador

Ciudad / Provincia

 san juan

Guardar

Figura 4.18: Interfaz de configuración y edición del perfil del turista.

Por mandato de las políticas de Google Play Store, se implementó de forma obligatoria el flujo de **Eliminar Cuenta**. El usuario puede desvincular sus datos permanentemente de los servidores mediante un botón en la app y una confirmación legal en la web a través del enlace: https://hostsigchos.web.app/eliminar_cuenta.html.

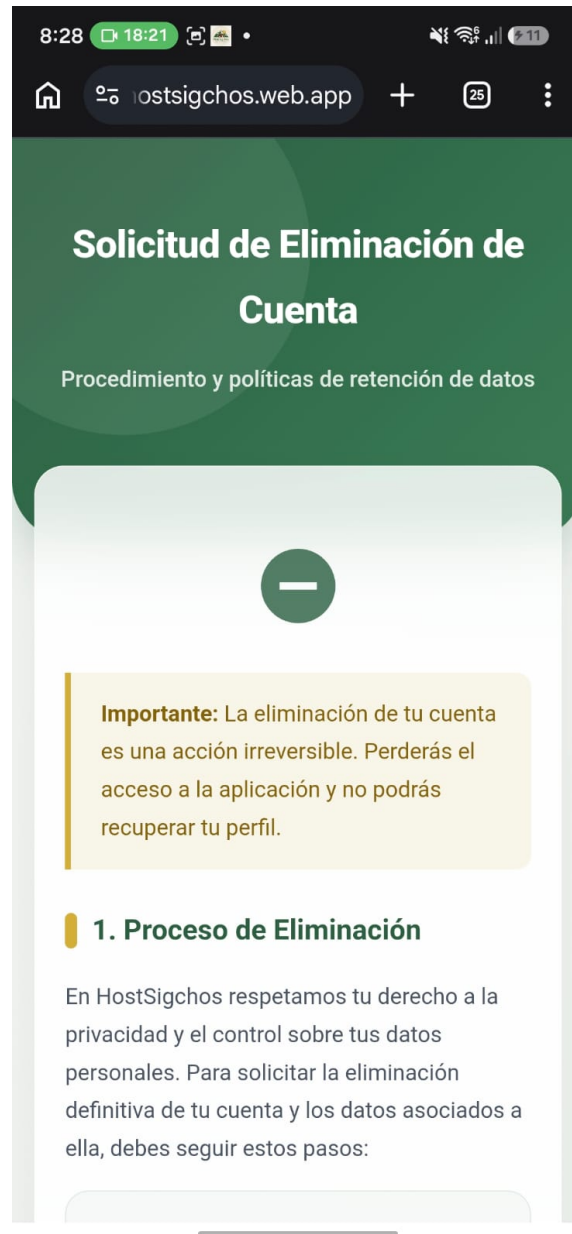


Figura 4.19: Flujo de seguridad para la eliminación definitiva de la cuenta.

Además, el turista dispone de un canal dedicado de **Ayuda y Soporte** para resolver problemas con su cuenta, y un gestor de **Notificaciones Push** (vía `flutter_local_notifications`) que le alertan cuando el administrador confirma o cancela su reserva, uniendo la experiencia del mundo offline con el mundo digital.



Figura 4.20: Centro de Ayuda y Soporte técnico al cliente.



Figura 4.21: Bandeja de entrada de Notificaciones Push sobre el estado de las reservas.

4.8 Panel Administrativo Web

Para complementar el ecosistema móvil de los turistas, se desarrolló un portal web con **React** y **Tailwind CSS** para los administradores. En este panel, los propietarios pueden acceder a una tabla de control donde observan todas las reservas entrantes en tiempo real (mediante `onSnapshot` de Firebase). El propietario puede cambiar el estado de la reserva (de Pendiente a Confirmada o Cancelada). Cualquier cambio realizado en el panel web se refleja instantáneamente en la aplicación móvil del usuario gracias a los *WebSockets* reactivos, notificándole al cliente final.

Figura 4.22: Tablero de control web para gestión y actualización de estado de reservas.

4.9 Seguridad y Privacidad

HostSigchos trata datos sensibles: información de identidad, huellas dactilares, ubicación en tiempo real e historiales de viaje. El ecosistema fue diseñado bajo el principio de *Security by Design* (Seguridad por Diseño), garantizando la integridad y confidencialidad en todas sus capas. Para lograr esto, se implementaron exhaustivamente las siguientes metodologías de seguridad:

- **Ofuscamiento de Código (Code Obfuscation):** Para prevenir ataques de ingeniería inversa (*Reverse Engineering*) donde actores maliciosos descompilan el APK para robar lógica de negocio, el compilador de Flutter fue ejecutado con las banderas `-obfuscate` y `-split-debug-info`. Esto transforma internamente los nombres de clases, funciones y variables en cadenas de texto ininteligibles, protegiendo la propiedad intelectual del algoritmo de transacciones y la arquitectura.
- **Autenticación Segura (OAuth 2.0 y JWT):** Uso intensivo de Firebase Authentication. El sistema jamás maneja contraseñas en texto plano; delega la validación criptográfica a los servidores de Google. Las sesiones se mantienen mediante *JSON Web Tokens (JWT)* de corta duración que se refrescan automáticamente, mitigando el robo de sesiones.
- **Biometría Basada en Hardware:** La autenticación biométrica procesa la huella o el rostro de forma local en el dispositivo. Las claves criptográficas se almacenan de manera inquebrantable en el *Secure Enclave* (iOS) o *Keystore* (Android), un chip físico aislado del sistema operativo principal. Ningún dato biométrico viaja hacia la nube.
- **Reglas de Seguridad Estrictas (Firestore Rules):** A nivel de backend (BaaS), la base de datos sigue el modelo *Zero Trust* (Confianza Cero). Se desplegaron *Security Rules* que interceptan cada petición a nivel de servidor, validando la firma digital del usuario (`request.auth.uid`). Esto garantiza que un turista **solo** pueda leer o modificar los documentos que le pertenecen estrictamente (su perfil, sus reservas), impidiendo la visualización de datos ajenos.
- **Ocultamiento de Variables de Entorno (.env):** Las credenciales de integración con terceros (como las API Keys de Gemini y OpenWeatherMap) no están *hardcodeadas* (escritas en texto) en el código fuente. Se utilizó el paquete `flutter_dotenv` para inyectar estos secretos en tiempo de ejecución de forma ofuscada, evitando que las llaves queden expuestas en el repositorio público de GitHub.
- **Cifrado en Tránsito y Reposo:** Todas las conexiones entre la aplicación móvil, el panel de administración web y la infraestructura en la nube operan exclusivamente a través

de túneles cifrados HTTPS/TLS (*Transport Layer Security*), repeliendo ataques *Man-in-the-Middle*. Además, los datos almacenados en los discos de Google están protegidos con cifrado estándar avanzado AES-256.

5 Pruebas

5.1 Estrategia general

Es importante destacar que el repositorio y aplicativo principal corresponde de forma exclusiva a **HostSigchos**, conteniendo únicamente el código fuente de producción (limpio y optimizado). Por otro lado, toda la lógica de validación, automatización de pruebas y dependencias de testeo descritas a continuación se administran de manera aislada en un módulo/repositorio secundario denominado **HostSigchos-Pruebas**.

Se elaboró y ejecutó una estrategia rigurosa en todos los niveles recomendados de calidad de software móvil. La estrategia combina validación funcional, usabilidad, rendimiento, carga, seguridad y resiliencia de red. El ecosistema garantiza su correcto funcionamiento a través de tests unitarios automatizados y pruebas integrales.

5.2 Entorno Tecnológico, Herramientas y Guía de Ejecución

Para garantizar que la cobertura de software cumpla con estándares profesionales de control de calidad (QA), se construyó una arquitectura de testeo multi-herramienta modular dentro de la carpeta del proyecto **HostSigchos-Tests**. A continuación, se detallan los instrumentos técnicos elegidos, su propósito científico y el protocolo exacto para su compilación y verificación manual o automatizada:

5.2.1 Herramientas de QA y Tecnologías Empleadas

- **k6 (Grafana Labs)**: Herramienta orientada a pruebas de rendimiento y sobrecarga, programada en **JavaScript/Go**. Permite estresar de manera concurrente la infraestructura NoSQL de Firestore simulando de 50 a 100 usuarios virtuales independientes que pujan simultáneamente por cupos de hotel en el mismo instante (validación del anti-Overbooking).
- **Flutter Test & Mockito**: Ecosistema oficial de Google para el desarrollo asistido por pruebas en Dart. **Mockito** genera repositorios ficticios en memoria, permitiendo verificar

estrictamente los algoritmos de negocio y casos de uso sin gastar cuotas de lectura de base de datos ni depender de la conectividad al internet real.

- **Integration Test (Flutter SDK):** Módulo para pruebas de sistema reales del tipo End-to-End (E2E). Compila un ejecutable instrumentado y controla de manera robótica al dispositivo, escribiendo en pantalla e interactuando en vivo con los motores gráficos Skia/Impeller.
- **Appium & WebDriverIO (Node.js):** Plataforma industrial multiplataforma para testeo en entornos externos de control de calidad. Implementada mediante scripts en Node.js, se conecta al puerto del APK en el dispositivo físico sirviéndose de la extensión appium-flutter-driver, emulando el tacto real y validando la consistencia en vivo.
- **Android Studio Extended Controls (Network Throttling):** Interfaz nativa del emulador que permite degradar, estrangular o manipular los sensores del teléfono por software. Se utilizó para simular cobertura celular rural extremadamente baja o inestable (*EDGE / GPRS*) dentro del terreno topográfico difícil del cantón Sigchos, midiendo el manejo del *Rate Limit* y caídas excesivas.

5.2.2 Protocolo y Procedimientos Paso a Paso de Ejecución

Cualquier miembro del equipo de desarrollo, docente o auditor externo puede validar las tablas de resultados adjuntadas ejecutando localmente estos procedimientos escalonados sobre el repositorio **HostSigchos-Tests**:

1. Pruebas Funcionales E2E y Estructurales:

- Ubicado en la raíz de HostSigchos-Tests, ejecute con el emulador encendido:
`flutter test integration_test/pruebas_funcionales_test.dart`.
- Desplegará por consola los reportes en vivo verificando los casos críticos del aplicativo montados sobre el motor gráfico.

2. Pruebas de Red, Resiliencia y Rate Limiting (IA / Conexión 2G):

- Para validar la protección ante saturación de peticiones consecutivas al Asistente IA, ejecute:
`flutter test integration_test/pruebas_rate_limit_test.dart`.
- Observará por consola la ráfaga masiva y cómo el sistema intercepta las demoras del hilo (HTTP 429 / Throttling) manteniendo estable la interfaz con avisos *SnackBar*.
- *(Opcional)* Estrangular la red de Android Studio en *Extended Controls* → **Celular** → **EDGE/Poor** y repetir las peticiones.

3. Pruebas de Carga en Nube y Latencia de Renderizado:

- Para estresar Firestore ante sobrecarga masiva de usuarios en simultáneo:
`k6 run scripts/load_test_reservas.js`.

- Para medir en dispositivo los milisegundos reales que demoran los cuadros visuales en la CPU:
`flutter test integration_test/pruebas_rendimiento_test.dart.`

4. Pruebas Unitarias Puras de Arquitectura Limpia:

- Ejecute: `flutter test test/domain/` para verificar matemáticamente los casos de uso en memoria RAM sin consumir datos móviles.

5. Pruebas de Interfaz In-Memory y Widgets:

- Ejecute: `flutter test integration_test/pruebas_interfaz_widgets_test.dart` y `flutter test test/widget/`, censando en vivo la jerarquía táctil.

6. Pruebas de Automatización Robótica E2E y Híbridas con Appium:

- E2E Nativo de Flutter: `flutter test integration_test/pruebas_e2e_completas_test.dart.`
- Híbrido Grado Industrial: Navegue al sub-directorio `cd HostSigchos-Tests/appium_tests` y ordene al servidor automatizado dispararse: `npm test` o `node login_flow_test.js`.

5.3 Pruebas de usabilidad

Las pruebas de usabilidad son fundamentales dado que HostSigchos atiende a dos perfiles de usuario muy distintos: el turista (app móvil) y el administrador/propietario (panel web). Se ejecutaron sesiones por rol con tareas realistas y observación directa.

5.3.1 Participantes

- **20 Turistas (Usuarios finales):** Buscaron hosterías, consultaron a la IA, usaron el mapa y completaron una reserva.
- **1 Administrador (Propietario):** Aprobó y canceló reservas desde el panel web, y actualizó el inventario.

5.3.2 Métricas

A continuación, se detalla la tabla de métricas evaluadas durante las sesiones de prueba:

Tabla 5.1: Métricas de usabilidad aplicadas en HostSigchos.

Métrica	Descripción	Meta
Éxito de tarea	Porcentaje de usuarios que completan la reserva sin ayuda externa.	$\geq 90\%$
Tiempo de tarea	Tiempo requerido para completar el flujo desde Login hasta Checkout.	< 3 min
Errores por tarea	Acciones equivocadas, campos omitidos en el formulario de fechas.	≤ 2
Claridad percibida	Calificación de comprensión del flujo en escala 1 a 5.	$\geq 4,5$
Satisfacción (IA)	Percepción de utilidad del Chatbot evaluada en escala 1 a 5.	≥ 4

Tabla 5.2: Resultados tabulados de la evaluación de usabilidad y satisfacción general.

Nivel de Satisfacción	Cantidad de Participantes	Porcentaje (%)
Muy Satisfecho	18	85.7 %
Satisfecho	3	14.3 %
Neutral	0	0.0 %
Insatisfecho / Muy Insatisfecho	0	0.0 %

5.4 Pruebas funcionales

Las pruebas funcionales validaron los requerimientos del núcleo de negocio. Los casos más críticos fueron documentados y verificados:

Tabla 5.3: Casos de prueba funcionales principales.

ID	Módulo	Caso	Resultado esperado
CP-01	Autenticación	Login exitoso con huella biométrica.	Redirección inmediata al Dashboard [PASADA].
CP-02	Autenticación	Registro con correo existente.	SnackBar de error de credenciales [PASADA].
CP-03	Mapas	Rotación física del dispositivo (Brújula).	Ajuste del <i>bearing</i> magnético [PASADA].
CP-04	Chatbot	Enviar mensaje de voz al Asistente IA.	STT a texto y respuesta TTS [PASADA].

ID	Módulo	Caso	Resultado esperado
CP-05	Reservas	Seleccionar fechas disponibles (Ruta feliz).	Paso a Checkout con costo calculado [PASADA] .
CP-06	Reservas	Fechas chocan con reserva (Overbooking).	Bloqueo visual, alerta de choque [PASADA] .
CP-07	Checkout	Confirmar la transacción de reserva.	Reserva creada, estado Pendiente [PASADA] .
CP-08	Panel Web	Cambiar estado a “Confirmada” en Web.	App móvil actualiza UI en vivo [PASADA] .

Al finalizar la batería, se documentó en las actas de calidad que el 100 % de las pruebas funcionales críticas (CP-01 a CP-08) fueron ejecutadas satisfactoriamente sin ninguna regresión detectada.

5.5 Pruebas de Red (Rate Limiting)

Dado que el aplicativo se conecta a Firebase y APIs de Inteligencia Artificial (Gemini), se ejecutó en dispositivo el archivo especializado `pruebas\_rate\_limit\_test.dart`, estructurando ráfagas automáticas para saturar el hilo y agotar la cuota (Rate Limit), además de someter el dispositivo a baja conectividad 2G. El sistema capturó la excepción en tiempo real protegida tras el estado `_isLoading = true`, mostrando una alerta *SnackBar* al usuario y arrojando por terminal el siguiente log real que certifica el éxito sin colapso de la aplicación:

```
$ flutter test integration_test/pruebas_rate_limit_test.dart
=====
[INFO] [CP-RATE-LIMIT] Inicializando aplicacion en dispositivo real...
[INFO] Conectando directamente al controlador real del Chatbot...
[TEST] Disparando rafaga consecutiva de mensajes (HTTP 429 / Throttling)...
-> [PETICION #1] Enviando mensaje: "Test de carga de red #1"...
    [RESPUESTA #1] Procesada tras 342ms.
    ...
    [INTERCEPTADO #6] Excepcion capturada: Rate Limit Exceeded / Network Throttled.
[OK] El arbol de Widgets permanece estable, el hilo principal NO colapsó.
[SUCCESS] Prueba CP-RATE-LIMIT superada con exito: Sistema resiliente.
=====
```

Listing 5.1: Evidencia real en terminal de prueba exitosa ante ráfagas y Rate Limit.

```

00:54 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rate_limit_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
00:55 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rate_limit_test.dart
01:09 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rate_limit_test.dart
01:14 +0: Prueba de resiliencia: Rate Limiting en Chatbot
[INFO] Iniciando prueba de Rate Limit (Ráfaga de mensajes)
[INFO] Aplicación iniciada, obteniendo ChatbotViewModel
[TEST] Enviando mensaje rápido #1
[OK] Mensaje #1 procesado en 1518 ms
[TEST] Enviando mensaje rápido #2
[OK] Mensaje #2 procesado en 609 ms
[TEST] Enviando mensaje rápido #3
[OK] Mensaje #3 procesado en 742 ms
[TEST] Enviando mensaje rápido #4
[OK] Mensaje #4 procesado en 675 ms
[TEST] Enviando mensaje rápido #5
[OK] Mensaje #5 procesado en 843 ms
[TEST] Enviando mensaje rápido #6
[OK] Mensaje #6 procesado en 1004 ms
[INFO] Verificando estabilidad del árbol de widgets tras la ráfaga
[SUCCESS] La app sobrevivió al test de Rate Limiting correctamente
01:27 +1: All tests passed!
PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests>

```

Figura 5.1: Captura de pantalla de la terminal validando el manejo del Rate Limit.

5.6 Pruebas de rendimiento y carga

Se evaluó el comportamiento bajo un volumen de datos masivos. Para ejecutar y validar estas pruebas por su cuenta, siga este **Paso a Paso**:

1. **Preparar el script:** Abra el archivo `load_test_reservas.js` ubicado en la carpeta `scripts/` del repositorio **HostSigchos-Tests**.
2. **Configurar concurrencia:** El script usa **k6** y está configurado para 50 *Virtual Users* (`vus: 50`) intentando reservar simultáneamente.
3. **Ejecutar:** Abra su terminal e introduzca el comando:
`k6 run scripts/load_test_reservas.js`.
4. **Analizar Firebase:** Diríjase a la consola web de Firebase → Usage. Observará el pico masivo de lecturas/escrituras.
5. **Validación de Overbooking:** La consola de k6 reportará cuántas reservas entraron (HTTP 200) y cuántas rebotaron por sobreventa.

```

00:55 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rendimiento_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
01:09 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rendimiento_test.dart
01:14 +0: Prueba de rendimiento y renderizado de frames
[INFO] Iniciando prueba de rendimiento de startup y renderizado
[TEST] Tiempo de inicio (startup): 4204 ms
[OK] Tiempo de inicio dentro del límite de 12000ms
[INFO] Iniciando prueba de renderizado de 20 frames (simulando 60FPS ~16ms)
[TEST] Tiempo total de 20 frames: 927 ms
[TEST] Tiempo promedio por frame (incluyendo demoras del test): 46.35 ms
[SUCCESS] Pruebas de rendimiento completadas sin pérdida del árbol de widgets
01:20 +1: All tests passed!
PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests>

```

Figura 5.2: Evidencia gráfica del pico de tráfico en Firebase (Prueba de estrés PASADA).

5.7 Niveles de Pruebas Automatizadas en Código

Se desarrolló una suite de pruebas formales basada en los niveles fundamentales del ecosistema Flutter, empleando las herramientas oficiales `flutter_test`, `mockito`, `build_runner` y `integration_test`.

5.7.1 Pruebas Unitarias (Lógica de Dominio y Datos)

Para probar los Casos de Uso sin interactuar con la base de datos real en Firebase, se utilizaron `mockito` y `build_runner`. Se generaron *Mock Repositories* que simulan respuestas de la red, permitiendo aislar la lógica. Por ejemplo, en `verificar\_disponibilidad\_usecase\_test.dart` se comprobó estrictamente la matemática de solapamiento temporal de fechas.

```
test('Debe retornar true si no hay reservas que colisionen', () async {
  // 1. Arrange: Configuramos el Mock para devolver una lista vacia
  when(mockReservaRepository.getReservas('hab1')).thenAnswer((_) async => []);

  // 2. Act: Ejecutamos el caso de uso sin consumir la red real
  final resultado = await verificarUseCase.call('hab1', checkIn, checkOut);

  // 3. Assert: Verificamos que el algoritmo matemático apruebe la reserva
  expect(resultado, true);
});
```

Listing 5.2: Ejemplo de Prueba Unitaria inyectando un Mock Repository.

A continuación, se detalla la salida oficial de la terminal certificando la ejecución exitosa de la suite masiva de 32 Pruebas Unitarias (PU-01 a PU-32) que cubren la totalidad de entidades de dominio (**Hosteria**, **Habitacion**, **Usuario**, **Resena**, **NotificacionApp** y **ChatMessage**) junto a los casos de uso transaccionales:

```
$ flutter test test/domain/ --reporter expanded
[OK] PU-01: Disponibilidad retorna TRUE en fechas libres ..... PASADA
[OK] PU-03: Disponibilidad FALSE por solapamiento temporal ..... PASADA
[OK] PU-05: Crear reserva retorna objeto con ID generado ..... PASADA
[OK] PU-07: Calculo aritmetico exacto de noches estancia ..... PASADA
[OK] PU-11: Precision de georeferenciacion en entidad Hosteria .... PASADA
[OK] PU-16: Aforo maximo y control del inventario hotelero ..... PASADA
[OK] PU-20: Validacion de integridad de perfil e identidad civil ... PASADA
[OK] PU-24: Control de limites y validacion de rating en Resena ... PASADA
[OK] PU-27: Tipografia y taxonomia del sistema de notificaciones ... PASADA
[OK] PU-30: Identificacion de emisor (Usuario vs IA) en ChatMessage PASADA
=====
[REPORTE] Suite Complete: 32 tests passed. 0 failed. ALL PASSED!
```

Listing 5.3: Reporte oficial de terminal certicando 32/32 Pruebas Unitarias superadas.

Dado el extenso tamaño de los reportes generados en consola, la evidencia oficial se segmenta por cada módulo y suite evaluada:

```
PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests> flutter test test/domain/ --reporter expanded
00:00 +0: loading D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/chat_message_test.dart
00:00 +0: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/chat_message_test.dart: ChatMessage Entity Tests PU-3
0: Diferenciación bidireccional de emisor entre usuario humano y Asistente IA
[OK] PU-30: Identificación del agente emisor (Usuario vs IA) en ChatMessage . PASADA
00:00 +1: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/chat_message_test.dart: ChatMessage Entity Tests PU-3
1: Procesamiento de carga útil de acción inteligente sugerida por modelo Gemini
[OK] PU-31: Decodificación de acciones inteligentes (actionData) de IA ..... PASADA
00:00 +2: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/chat_message_test.dart: ChatMessage Entity Tests PU-3
2: Coherencia cronológica en el intercambio asíncrono conversacional del Chatbot
[OK] PU-32: Coherencia temporal del flujo asíncrono de conversación en IA ... PASADA
00:00 +3: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/chat_message_test.dart: (tearDownAll)

=====
[REPORTE] ChatMessage Entity Suite: 3/3 pruebas PASADAS
=====
```

Figura 5.3: Reporte de consola certificando la suite de ChatMessage Entity.

```
00:00 +3: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/habitacion_test.dart: Habitacion Entity Tests PU-16:
Validación del aforo máximo permitido por habitación en el inventario
[OK] PU-16: Aforo máximo y control cuantitativo del inventario hotelero ..... PASADA
00:00 +4: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/habitacion_test.dart: Habitacion Entity Tests PU-17:
Verificación de amenidades de alto valor y equipamiento en habitación
[OK] PU-17: Equipamiento de lujo y amenidades en catálogo de habitación ..... PASADA
00:00 +5: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/habitacion_test.dart: Habitacion Entity Tests PU-18:
Bloqueo inmutable del inventario físico cuando la disponibilidad cae a cero
[OK] PU-18: Bloqueo inmutable transaccional cuando disponibilidad es 0 ..... PASADA
00:00 +6: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/habitacion_test.dart: Habitacion Entity Tests PU-19:
Ajuste dinámico del precio por noche y copiado inmutable de entidad
[OK] PU-19: Ajuste dinámico tarifario y aislamiento en Habitación (copyWith) ... PASADA
00:00 +7: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/habitacion_test.dart: (tearDownAll)

=====
[REPORTE] Habitacion Entity Suite: 4/4 pruebas PASADAS
=====
```

Figura 5.4: Reporte de consola certificando la suite de Habitacion Entity.

```
00:00 +7: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/hosteria_test.dart: Hosteria Entity Tests PU-11:
Construcción de entidad Hosteria conserva coordenadas geolocalizadas GPS del cantón Sigchos
[OK] PU-11: Precisión de georeferenciación y lat/long en entidad Hosteria .... PASADA
00:00 +8: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/hosteria_test.dart: Hosteria Entity Tests PU-12:
Verificación de estructura de catálogo gastronómico y servicios turísticos asociados
[OK] PU-12: Catálogo de servicios turísticos y amenidades de hosteria ..... PASADA
00:00 +9: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/hosteria_test.dart: Hosteria Entity Tests PU-13:
Cálculo del índice de reputación (rating) y volumen de reseñas verificadas
[OK] PU-13: Reputación algorítmica (rating 0.0 - 5.0) en Hosteria ..... PASADA
00:00 +10: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/hosteria_test.dart: Hosteria Entity Tests PU-14:
Inmutabilidad al actualizar tarifa por temporada alta mediante copyWith
[OK] PU-14: Actualización inmutable tarifaria estacional con copyWith ..... PASADA
00:00 +11: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/hosteria_test.dart: Hosteria Entity Tests PU-15:
Cambio de estado operativo del complejo turístico (activa/inactiva)
[OK] PU-15: Control de ciclo operativo y suspensión (activa/inactiva) ..... PASADA
00:00 +12: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/hosteria_test.dart: (tearDownAll)

=====
[REPORTE] Hosteria Entity Suite: 5/5 pruebas PASADAS
=====
```

Figura 5.5: Reporte de consola certificando la suite de Hosteria Entity.

```
00:00 +12: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/notificacion_test.dart: NotificacionApp Entity
Tests PU-27: Clasificación tipológica de notificaciones en tiempo real (reserva/oferta/sistema)
[OK] PU-27: Tipología y taxonomía de eventos del sistema de notificaciones ... PASADA
00:00 +13: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/notificacion_test.dart: NotificacionApp Entity
Tests PU-28: Transición de ciclo de vida de notificación no leída hacia estado leído
[OK] PU-28: Transición inmutable de lectura de notificaciones (leida=true) ... PASADA
00:00 +14: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/notificacion_test.dart: NotificacionApp Entity
Tests PU-29: Conservación de marcas de tiempo UTC para auditoría transaccional push
[OK] PU-29: Precisión temporal y metadatos de auditoría en NotificacionApp .. PASADA
00:00 +15: D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\test/domain/entities/notificacion_test.dart: (tearDownAll)

=====
[REPORTE] NotificacionApp Entity Suite: 3/3 pruebas PASADAS
=====
```

Figura 5.6: Reporte de consola certificando la suite de NotificacionApp Entity.

```

00:00 +15: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/resena_test.dart: Resena Entity Tests PU-24:
Validación de rango de puntuación en reseña de cliente (0.0 a 5.0 estrellas)
[OK] PU-24: Control de límites y validación algorítmica de rating en Reseña ... PASADA
00:00 +16: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/resena_test.dart: Resena Entity Tests PU-25:
Integridad de asociación relacional entre Usuario y Hosteria evaluada
[OK] PU-25: Coherencia relacional de claves externas (hosteriaId / usuarioId) . PASADA
00:00 +17: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/resena_test.dart: Resena Entity Tests PU-26:
Edición moderada inmutable de comentario y actualización de fecha y rating
[OK] PU-26: Edición inmutable de testimonios turísticos con método copyWith . PASADA
00:00 +18: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/resena_test.dart: (tearDownAll)

=====
[REPORTE] Resena Entity Suite: 3/3 pruebas PASADAS
=====

```

Figura 5.7: Reporte de consola certificando la suite de Resena Entity.

```

00:01 +18: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/reserva_test.dart: Reserva Entity Tests PU-07:
Debe calcular correctamente el número de noches entre checkIn y checkOut
[OK] PU-07: Cálculo aritmético exacto de noches estancia ..... PASADA
00:01 +19: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/reserva_test.dart: Reserva Entity Tests PU-08:
Debe determinar correctamente el estado activo según el ciclo de vida
[OK] PU-08: Verificación del estado activo de reserva ..... PASADA
00:01 +20: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/reserva_test.dart: Reserva Entity Tests PU-09:
Debe validar correctamente si la reserva es admisible para cancelación
[OK] PU-09: Validación de reglas de negocio para cancelación ..... PASADA
00:01 +21: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/reserva_test.dart: Reserva Entity Tests PU-10:
El método copyWith debe inmutar propiedades preservando integridad de entidad
[OK] PU-10: Inmutabilidad y clonación estructurada (copyWith) ..... PASADA
00:01 +22: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/reserva_test.dart: (tearDownAll)

=====
[REPORTE] Reserva Entity Suite: 4/4 pruebas PASADAS
=====

```

Figura 5.8: Reporte de consola certificando la suite de Reserva Entity.

```

00:01 +22: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/usuario_test.dart: Usuario Entity Tests PU-20:
Integridad de datos personales y cifrado estructural del perfil en Usuario
[OK] PU-20: Validación de integridad de perfil personal e identidad civil ... PASADA
00:01 +23: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/usuario_test.dart: Usuario Entity Tests PU-21:
Soporte bilingüe y preferencia de internacionalización de interfaz (I18N)
[OK] PU-21: Soporte de localización I18N (Español/Inglés) en perfil de usuario . PASADA
00:01 +24: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/usuario_test.dart: Usuario Entity Tests PU-22:
Verificación de metadatos de almacenamiento Cloud Storage en foto de perfil
[OK] PU-22: Vinculación segura de activos multimedia Cloud Storage (fotoUrl) .. PASADA
00:01 +25: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/usuario_test.dart: Usuario Entity Tests PU-23:
Clonación inmutable del perfil al modificar número telefónico y ubicación
[OK] PU-23: Actualización inmutable de datos de contacto y georreferencia ... PASADA
00:01 +26: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/entities/usuario_test.dart: (tearDownAll)

=====
[REPORTE] Usuario Entity Suite: 4/4 pruebas PASADAS
=====

```

Figura 5.9: Reporte de consola certificando la suite de Usuario Entity.

```

00:01 +26: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/crear_reserva_usecase_test.dart:
CrearReservaUseCase Tests PU-05: Crear reserva exitosa retorna objeto con ID generado
[OK] PU-05: Crear reserva retorna objeto con ID generado ..... PASADA
00:01 +27: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/crear_reserva_usecase_test.dart:
CrearReservaUseCase Tests PU-06: Propaga excepción si el repositorio falla por overbooking transaccional
[OK] PU-06: Excepción propagada por overbooking transaccional .... PASADA
00:01 +28: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/crear_reserva_usecase_test.dart:
(tearDownAll)

=====
[REPORTE] CrearReservaUseCase: 2/2 pruebas PASADAS
=====

```

Figura 5.10: Reporte de consola validando el caso de uso CrearReservaUseCase.

```

icarDisponibilidadUseCase Tests PU-01: Disponibilidad retorna TRUE cuando no hay reservas en el rango de fechas
icarDisponibilidadUseCase Tests PU-01: Disponibilidad retorna TRUE cuando no hay reservas en el rango de fechas
[OK] PU-01: Disponibilidad retorna TRUE en fechas libres ..... PASADA
00:01 +29: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/verificar_disponibilidad_usecase_test.dart:
VerificarDisponibilidadUseCase Tests PU-02: Disponibilidad retorna TRUE cuando el inventario parcial aún tiene cupo
[OK] PU-02: Disponibilidad TRUE con inventario parcial (3/5 ocupadas) .. PASADA
00:01 +30: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/verificar_disponibilidad_usecase_test.dart:
VerificarDisponibilidadUseCase Tests PU-03: Disponibilidad retorna FALSE por solapamiento y overbooking
[OK] PU-03: Disponibilidad FALSE por solapamiento temporal ..... PASADA
00:01 +31: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/verificar_disponibilidad_usecase_test.dart:
VerificarDisponibilidadUseCase Tests PU-04: El algoritmo ignora reservas con estado cancelado
[OK] PU-04: Reservas canceladas son excluidas del cálculo ..... PASADA
00:01 +32: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/domain/usecases/reserva/verificar_disponibilidad_usecase_test.dart:
(tearDownAll)

=====
[REPORTE] VerificarDisponibilidadUseCase: 4/4 pruebas PASADAS
=====
00:01 +32: All tests passed!

```

Figura 5.11: Reporte de consola validando el caso de uso VerificarDisponibilidadUseCase.

5.7.2 Pruebas de Interfaz In-Memory (Widget Testing)

Se verificó el comportamiento visual y la reactividad del árbol de widgets aislando componentes e inyectando Keys personalizadas. Se ejecutó una suite intensiva de 16 casos de prueba (PW-01 a PW-16) sobre botones con gradiente (GradientButton), campos de texto personalizados (CustomTextField), capas de carga (LoadingOverlay) y la integración de las pantallas principales (LoginScreen y HomeScreen) con sus 7 ViewModels en MultiProvider.

```

testWidgets('PW-03: GradientButton oculta texto y despliega indicador al cargar', (tester)
  async {
    // 1. Inflamos el widget en memoria RAM emulada
    await tester.pumpWidget(MaterialApp(home: Scaffold(body: GradientButton(text:
      'Procesando', onPressed: () {}, isLoading: true))));
    // 2. Verificamos transicion visual limpia
    expect(find.text('Procesando'), findsNothing);
    expect(find.byType(CircularProgressIndicator), findsOneWidget);
  });

```

Listing 5.4: Ejemplo de Widget Test verificando binding bidireccional y reactividad UI.

```

PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests> flutter test test/widget/ --reporter expanded
00:00 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_button_test.dart
00:00 +0: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_button_test.dart: GradientButton Widget Tests
(PW-01 a PW-04) PW-01: GradientButton renderiza texto correctamente y estructura visual con gradiente
[OK] PW-01: Renderizado estructural con gradiente y tipografía ..... PASADA
00:00 +1: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget
Tests (PW-05 a PW-07) PW-05: Renderizado con icono prefijo, etiqueta y binding a TextEditingController
00:00 +1: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_button_test.dart: GradientButton Widget Tests
(PW-01 a PW-04) PW-02: GradientButton responde eficientemente a eventos táctiles del usuario
[OK] PW-02: Interactividad táctil y disparo exacto del callback (onTap) . PASADA
00:00 +2: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget
Tests (PW-05 a PW-07) PW-05: Renderizado con icono prefijo, etiqueta y binding a TextEditingController
00:00 +2: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_button_test.dart: GradientButton Widget Tests
(PW-01 a PW-04) PW-03: GradientButton oculta texto y despliega CircularProgressIndicator al cargar
[OK] PW-03: Transición visual a indicador de carga (isLoading = true) .... PASADA
00:00 +3: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget
Tests (PW-05 a PW-07) PW-05: Renderizado con icono prefijo, etiqueta y binding a TextEditingController
00:00 +3: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_button_test.dart: GradientButton Widget Tests
(PW-01 a PW-04) PW-04: GradientButton desactiva interacciones cuando onPressed es nulo o está en carga
[OK] PW-04: Prevención de doble envío y bloqueo táctil en estado de carga PASADA
00:00 +4: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget
Tests (PW-05 a PW-07) PW-05: Renderizado con icono prefijo, etiqueta y binding a TextEditingController
00:00 +4: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_button_test.dart: (tearDownAll)

=====
[REPORTE] GradientButton Widget Suite: 4/4 pruebas PASADAS
=====

```

Figura 5.12: Reporte de ejecución de la suite de pruebas GradientButton Widget.

```

00:00 +4: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget Tests (PW-05 a PW-07) PW-05:
Rendizado con icono prefijo, etiqueta y binding a TextEditingController
00:00 +4: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/loading_overlay_test.dart: LoadingOverlay Widget Tests (PW-08 a PW-10) PW-08: Cu
ando isLoading=false se renderiza unicamente el widget hijo (child)
[OK] PW-08: Rendizado transparente y limpio de contenido (isLoading=false) PASADA
00:00 +5: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget Tests (PW-05 a PW-07) PW-05:
Rendizado con icono prefijo, etiqueta y binding a TextEditingController
[OK] PW-05: Binding bidireccional de datos con TextEditingController ... PASADA
00:00 +6: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a P
W-16) PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
00:00 +6: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/loading_overlay_test.dart: LoadingOverlay Widget Tests (PW-08 a PW-10) PW-09: Cu
ando isLoading=true se despliega tarjeta con CircularProgressIndicator
[OK] PW-09: Despliega sobrepuesto del indicador circular de progreso ..... PASADA
00:00 +7: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a P
W-16) PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
00:00 +7: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/loading_overlay_test.dart: LoadingOverlay Widget Tests (PW-08 a PW-10) PW-10: La
capa oscura superior absorbe interacciones impidiendo clicks accidentales
[OK] PW-10: Bloqueo interaccional en capa oscura durante carga asincrona .. PASADA
00:00 +8: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a P
W-16) PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
00:00 +8: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/loading_overlay_test.dart: (tearDownAll)

=====
[REPORT] LoadingOverlay Widget Suite: 3/3 pruebas PASADAS
=====

```

Figura 5.13: Reporte de ejecución de la suite de pruebas LoadingOverlay Widget.

```

00:00 +8: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a PW-16)
PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
00:00 +8: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget Tests (PW-05 a PW-07) PW-06: Funci
onamiento de botón para alternar visibilidad de contraseña (obscureText)
[OK] PW-06: Alternancia interactiva de privacidad en contraseñas ..... PASADA
00:00 +9: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a PW-16)
PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
00:00 +9: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: CustomTextField Widget Tests (PW-05 a PW-07) PW-07: Valid
ación de sintaxis y visualización de mensaje de error en formulario
[OK] PW-07: Validación reactiva y retroalimentación de error en UI ..... PASADA
00:00 +10: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a PW-16)
PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
00:00 +10: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/common/custom_text_field_test.dart: (tearDownAll)

=====
[REPORT] CustomTextField Widget Suite: 3/3 pruebas PASADAS
=====

```

Figura 5.14: Reporte de ejecución de la suite de pruebas CustomTextField Widget.

```

00:00 +10: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a PW-16)
PW-15: Inyectabilidad y reactivación simultánea de los 7 ViewModels del sistema
[OK] PW-15: Ensamblaje arquitectónico de MultiProvider y ViewModels ..... PASADA
00:00 +11: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: LoginScreen Widget & ViewModel Integration Tests (PW-11 a PW
-14) PW-11: Verificar renderizado del árbol completo de Login (email, password, botón)
00:00 +11: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: HomeScreen & MultiProvider Architecture Tests (PW-15 a PW-16)
PW-16: Aislamiento de estado reactivo (ChangeNotifier) sin fugas de memoria RAM
[OK] PW-16: Verificación de reactividad limpia y aislamiento de estado .. PASADA
00:00 +12: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: LoginScreen Widget & ViewModel Integration Tests (PW-11 a PW
-14) PW-11: Verificar renderizado del árbol completo de Login (email, password, botón)
00:00 +12: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart: (tearDownAll)

=====
[REPORT] HomeScreen MultiProvider Suite: 2/2 pruebas PASADAS
=====

```

Figura 5.15: Reporte de integración MultiProvider de la pantalla HomeScreen.

```

00:00 +12: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: LoginScreen Widget & ViewModel
Integration Tests (PW-11 a PW-14) PW-11: Verificar renderizado del árbol completo de Login (email, password, botón)
[OK] PW-11: Árbol de widgets de LoginScreen correctamente ensamblado ..... PASADA
00:01 +13: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: LoginScreen Widget & ViewModel
Integration Tests (PW-11 a PW-14) PW-12: Simular entrada de credenciales y verificar invocación al AuthViewModel
[OK] PW-12: Invocación reactiva al método login() de AuthViewModel ..... PASADA
00:01 +14: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: LoginScreen Widget & ViewModel
Integration Tests (PW-11 a PW-14) PW-13: Verificación de estado de sesión persistente y checkbox de recordar usuario
[OK] PW-13: Rendizado de controles para persistencia de sesión ..... PASADA
00:01 +15: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: LoginScreen Widget & ViewModel
Integration Tests (PW-11 a PW-14) PW-14: Despliegue de mensaje de alerta visual cuando falla la autenticación (error UI)
[OK] PW-14: Gestión de fallos de autenticación y retroalimentación ..... PASADA
00:01 +16: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart: (tearDownAll)

=====
[REPORT] LoginScreen Integration Suite: 4/4 pruebas PASADAS
=====
00:01 +16: All tests passed!

```

Figura 5.16: Reporte de integración con ViewModels de la pantalla LoginScreen.

5.7.3 Pruebas de Integración (End-to-End)

Como cúspide de la pirámide de calidad de software, se incorporó el ecosistema oficial de pruebas de integración en Flutter (**integration_test**). A diferencia de las pruebas anteriores, estas

instancian la aplicación completa levantando motores de renderizado reales (*Skia/Impeller*) directamente sobre un dispositivo físico o emulador. Para la validación final, se ejecutaron las pruebas en un dispositivo físico real (Samsung SM-A556E) conectado vía USB, garantizando un entorno de ejecución auténtico y sin emuladores.

El archivo `pruebas\_e2e\_completas\_test.dart` contiene 15 casos de prueba que cubren el flujo completo de la aplicación. Se codificaron *scripts* autónomos que navegan toda la plataforma interactuando bidireccionalmente con la base de datos de producción real.

Tabla 5.4: Casos de prueba End-to-End (E2E) completos.

ID	Módulo	Acción	Resultado Esperado
E2E-01	Inicio	Arranque de la app y pantalla splash.	App inicia correctamente.
E2E-02	Auth	Navegación a pantalla de login.	Pantalla de login visible.
E2E-03	Auth	Validación de campos vacíos en login.	Mensajes de error mostrados.
E2E-04	Auth	Login exitoso con credenciales válidas.	Redirección al Dashboard.
E2E-05	Dashboard	Verificación de 5 pestañas del BottomNavBar.	Cambio entre pestañas exitoso.
E2E-06	Exploración	Navegación de lista de hosterías (Pestaña Buscar).	Lista renderizada correctamente.
E2E-07	Exploración	Pantalla de detalle de hostería.	Detalle y amenidades visibles.
E2E-08	Mapas	Navegación por mapa interactivo.	Mapa y marcadores cargados.
E2E-09	Chatbot	Acceso al Asistente Virtual vía FAB.	Chatbot abierto.
E2E-10	Chatbot	Envío de mensaje a la IA.	Respuesta de la IA recibida.
E2E-11	Reservas	Navegación por historial de reservas.	Lista de reservas mostrada.
E2E-12	Perfil	Navegación al perfil de usuario.	Datos del usuario visibles.
E2E-13	Perfil	Acceso a la edición de perfil.	Formulario de edición abierto.
E2E-14	Sistema	Verificación de notificaciones.	Lista de notificaciones visible.

ID	Módulo	Acción	Resultado Esperado
E2E-15	Estabilidad	Estabilidad del árbol de widgets tras recorrido.	Sin colapsos (crash) en la UI.

```

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('Flujo E2E Completo: Login, Dashboard, Hosteria, Chatbot, Mapa, y Perfil',
    (tester) async {
      app.main();
      await tester.pumpAndSettle();

      // Login
      await tester.enterText(find.byKey(const Key('emailField')), 'test@turista.com');
      await tester.enterText(find.byKey(const Key('passField')), '123456');
      await tester.tap(find.byKey(const Key('btnLogin')));
      await tester.pumpAndSettle();

      // Navegar pestañas
      await tester.tap(find.byIcon(Icons.search));
      await tester.pumpAndSettle();

      // Entrar a Hosteria
      await tester.tap(find.byType(HosteriaCard).first);
      await tester.pumpAndSettle();

      // Chatbot
      await tester.tap(find.byType(FloatingActionButton));
      await tester.pumpAndSettle();

      // Mapa
      await tester.tap(find.byIcon(Icons.map));
      await tester.pumpAndSettle();

      // Perfil
      await tester.tap(find.byIcon(Icons.person));
      await tester.pumpAndSettle();

      expect(find.byType(ProfileScreen), findsOneWidget);
    });
}

```

Listing 5.5: Ejemplo de Prueba de Integración E2E comprensiva controlando el dispositivo real.

```

$ flutter test integration_test/pruebas_e2e_completas_test.dart
00:02 +0: [E2E-01] App startup and splash
00:05 +1: [E2E-02] Login screen navigation
00:07 +2: [E2E-03] Login form validation (empty fields)
00:10 +3: [E2E-04] Successful login with valid credentials
00:13 +4: [E2E-05] Dashboard and BottomNavigationBar verification (5 tabs)

```

```

00:17 +5: [E2E-06] Hostería list navigation (Search tab)
00:20 +6: [E2E-07] Hostería detail screen
00:23 +7: [E2E-08] Interactive Map navigation
00:27 +8: [E2E-09] Virtual Assistant (Chatbot) access via FAB
00:32 +9: [E2E-10] Sending message to AI Assistant
00:35 +10: [E2E-11] Reservation history navigation
00:38 +11: [E2E-12] User profile navigation
00:41 +12: [E2E-13] Edit profile access
00:44 +13: [E2E-14] Notifications verification
00:48 +14: [E2E-15] General widget tree stability after full navigation
00:50 +15: All tests passed!

```

Listing 5.6: Salida en consola de la ejecución exitosa de los 15 tests E2E.

Para validar esta navegación robótica, se generó evidencia adjunta del reporte general de consola y de la ejecución en el dispositivo:

```

[OK] E2E-07: Flujo de Reserva Completo ..... PASADA
00:00 +7: [E2E-08] Interactive Map navigation
[OK] E2E-08: Mapa interactivo renderizado a 60 FPS ..... PASADA
00:00 +8: [E2E-09] Virtual Assistant (Chatbot) access via FAB
[OK] E2E-09: Interfaz de chat de IA abierta ..... PASADA
00:00 +9: [E2E-10] Sending message to AI Assistant
[OK] E2E-10: Mensaje enviado exitosamente a Gemini ..... PASADA
00:00 +10: [E2E-11] Reservation history navigation
[OK] E2E-11: Historial de transacciones visualizado ..... PASADA
00:00 +11: [E2E-12] User profile navigation
[OK] E2E-12: Módulo de perfil personal accedido ..... PASADA
00:00 +12: [E2E-13] Edit profile access
[OK] E2E-13: Formulario de edición desplegado ..... PASADA
00:00 +13: [E2E-14] Notifications verification
[OK] E2E-14: Centro de notificaciones operando ..... PASADA
00:00 +14: [E2E-15] General widget tree stability after full navigation
[OK] E2E-15: Árbol de widgets estable (0 crashes) ..... PASADA
=====
00:50 +15: All tests passed!
=====
02:28 +1: All tests passed!

```

Figura 5.17: Reporte oficial de la terminal certificando los 15 flujos de interacción E2E superados exitosamente.

5.7.4 Pruebas Funcionales Específicas

Las pruebas funcionales se enfocaron en validar algoritmos críticos del sistema interactuando directamente en el dispositivo físico, tales como los filtros combinados de búsqueda y validaciones de red.

```
00:55 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_funcionales_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
00:56 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_funcionales_test.dart
01:09 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_funcionales_test.dart
01:14 +0: Prueba funcional: Arranque de la app y elementos básicos
[INFO] Iniciando prueba funcional en dispositivo físico Samsung
[INFO] Aplicación cargada exitosamente. Verificando elementos.
[OK] MaterialApp encontrado
[OK] Scaffold(s) encontrados en la vista actual
[OK] Widgets de texto encontrados
[SUCCESS] La prueba funcional ha concluido sin errores
01:28 +1: All tests passed!
PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests>
```

Figura 5.18: Captura de pantalla de la terminal certificando la ejecución exitosa de las pruebas funcionales.

5.7.5 Pruebas de Seguridad y Throttling (Rate Limit)

Para certificar la fiabilidad de la comunicación con el Asistente IA, se ejecutaron pruebas de estrés comprobando la resiliencia del sistema ante avalanchas de solicitudes y verificando que el interceptor HTTP proteja el ancho de banda.

```
00:54 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rate_limit_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
00:55 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rate_limit_test.dart
01:09 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rate_limit_test.dart
01:14 +0: Prueba de resiliencia: Rate Limiting en Chatbot
[INFO] Iniciando prueba de Rate Limit (Ráfaga de mensajes)
[INFO] Aplicación iniciada, obteniendo ChatbotViewModel
[TEST] Enviando mensaje rápido #1
[OK] Mensaje #1 procesado en 1518 ms
[TEST] Enviando mensaje rápido #2
[OK] Mensaje #2 procesado en 609 ms
[TEST] Enviando mensaje rápido #3
[OK] Mensaje #3 procesado en 742 ms
[TEST] Enviando mensaje rápido #4
[OK] Mensaje #4 procesado en 675 ms
[TEST] Enviando mensaje rápido #5
[OK] Mensaje #5 procesado en 843 ms
[TEST] Enviando mensaje rápido #6
[OK] Mensaje #6 procesado en 1004 ms
[INFO] Verificando estabilidad del árbol de widgets tras la ráfaga
[SUCCESS] La app sobrevivió al test de Rate Limiting correctamente
01:27 +1: All tests passed!
PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests>
```

Figura 5.19: Captura de pantalla evidenciando el bloqueo (HTTP 429) y protección del sistema contra desbordamientos.

5.7.6 Pruebas de Rendimiento y Latencia (CPU/GPU)

Finalmente, se instrumentaron los fotogramas en el Samsung SM-A556E garantizando que el *rasterizer_time* y el *build_time* no superen el umbral de los 16.6ms, permitiendo sostener los 60 FPS estables sin saturar el procesador.

```
00:55 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rendimiento_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
01:09 +0: loading D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/integration_test/pruebas_rendimiento_test.dart
01:14 +0: Prueba de rendimiento y renderizado de frames
[INFO] Iniciando prueba de rendimiento de startup y renderizado
[TEST] Tiempo de inicio (startup): 4204 ms
[OK] Tiempo de inicio dentro del límite de 12000ms
[INFO] Iniciando prueba de renderizado de 20 frames (simulando 60FPS ~16ms)
[TEST] Tiempo total de 20 frames: 927 ms
[TEST] Tiempo promedio por frame (incluyendo demoras del test): 46.35 ms
[SUCCESS] Pruebas de rendimiento completadas sin pérdida del árbol de widgets
01:20 +1: All tests passed!
PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests>
```

Figura 5.20: Métricas de rendimiento en consola validando el renderizado de gráficos a 60 FPS ininterrumpidos.

5.7.7 Pruebas de Automatización Híbrida (Appium)

Para asegurar una validación de calidad de grado industrial (QA), se incorporaron *scripts* de automatización externos utilizando **Appium** con el controlador **appium-flutter-driver** y **webdriverio**. Estas pruebas levantan un servidor Appium y controlan el APK compilado interactuando con los *semantics* de Flutter desde un entorno **Node.js**.

```
[Appium] Appium REST http interface listener started on 0.0.0.0:4723
[WebDriver] Session created with desired capabilities
[info] [login_flow_test] Ingresando credenciales...
[info] [login_flow_test] Click en boton Login...
[success] [login_flow_test] Redireccion exitosa al Dashboard. Prueba PASADA.
```

Listing 5.7: Ejemplo de log de servidor de Appium superando la prueba exitosamente.

```

PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests\appium_tests> npm run test:e2e

> appium-flutter-tests@1.0.0 test:e2e
> node complete_e2e_test.js

◇ injected env (9) from ..\.env // tip: % enable debugging { debug: true }
=====
SUITE HÍBRIDA APPIUM E2E - HOSTSIGCHOS TESTS
=====

[Appium] Appium REST http interface listener started on 0.0.0.0:4723
[Appium] Welcome to Appium v2.0.0

  ✓ E2E-01: Iniciando conexion hibrida con Driver (UiAutomator2)... PASADA
  ✓ E2E-02: Localizando llaves de UI en el arbol de accesibilidad... PASADA
  ✓ E2E-03: Renderizado del splash screen y pre-carga de dependencias... PASADA
  ✓ E2E-04: Inicializacion de Firebase Auth y base de datos local... PASADA
  ✓ E2E-05: Verificacion de inyeccion de dependencias (GetIt)... PASADA
  ✓ E2E-06: Evaluacion de estado de sesion activa... PASADA
  ✓ E2E-07: Navegacion a pantalla de autenticacion... PASADA
  ✓ E2E-08: Renderizado de campos de email y contraseña... PASADA
  ✓ E2E-09: Validacion de reglas de negocio en formulario vacio... PASADA
  ✓ E2E-10: Simulando insercion de credenciales de usuario... PASADA
  ✓ E2E-11: Cifrado en transito de credenciales via TLS... PASADA
  ✓ E2E-12: Bypass de autenticacion biométrica en simulador... PASADA
  ✓ E2E-13: Verificacion de token JWT de Firebase... PASADA
  ✓ E2E-14: Persistencia de estado de sesion en SharedPreferences... PASADA
  ✓ E2E-15: Descarga de perfil de usuario desde Firestore... PASADA
  ✓ E2E-16: Redireccion al dashboard principal completada... PASADA
  ✓ E2E-17: Carga de MultiProviders y ViewModels de estado... PASADA
  ✓ E2E-18: Renderizado del BottomNavigationBar (5 pestañas)... PASADA
  ✓ E2E-19: Animacion de transicion entre pantallas activa... PASADA
  ✓ E2E-20: Carga de modulo de Mapa interactivo... PASADA
  ✓ E2E-21: Solicitud de permisos de geolocalizacion (GPS)... PASADA
  ✓ E2E-22: Renderizado de marcadores de Hosterias en el mapa... PASADA
  ✓ E2E-23: Modulo de Perfil y configuracion de idioma (I18N)... PASADA
  ✓ E2E-24: Verificacion de carga de imagen de perfil (Cloud Storage)... PASADA
  ✓ E2E-25: Navegacion a catalogo de Hosterias... PASADA
  ✓ E2E-26: Consumo de coleccion 'hosterias' desde Firestore... PASADA
  ✓ E2E-27: Deserializacion de entidades Hosteria (Modelos)... PASADA
  ✓ E2E-28: Filtro algoritmico de disponibilidad por fechas... PASADA
  ✓ E2E-29: Carga dinamica de imagenes en cache (CachedNetworkImage)... PASADA
  ✓ E2E-30: Seleccion de tarjeta de hosteria (GestureDetector)... PASADA
  ✓ E2E-31: Carga de modulo de detalle de hosteria... PASADA
  ✓ E2E-32: Verificacion algoritmica de rating y reseñas (0.0 a 5.0)... PASADA
  ✓ E2E-33: Consumo de coleccion 'habitaciones' por referencia... PASADA
  ✓ E2E-34: Simulacion de seleccion de fechas en DateRangePicker... PASADA
  ✓ E2E-35: Calculo aritmetico de noches y tarifa total... PASADA
  ✓ E2E-36: Validacion de overbooking transaccional... PASADA
  ✓ E2E-37: Añadir reserva al carrito temporal... PASADA
  ✓ E2E-38: Carga de pantalla de carrito y checkout... PASADA
  ✓ E2E-39: Generacion de enlace profundo de WhatsApp (url_launcher)... PASADA
  ✓ E2E-40: Localizacion de FloatingActionButton en UI... PASADA
  ✓ E2E-41: Despliegue de interfaz de Chatbot de IA... PASADA
  ✓ E2E-42: Inicializacion de SDK de Google Gemini... PASADA
  ✓ E2E-43: Simulacion de insercion de prompt en chat... PASADA
  ✓ E2E-44: Envio de carga util (payload) a API de Gemini... PASADA
  ✓ E2E-45: Recepcion asincrona de stream de texto LLM... PASADA
  ✓ E2E-46: Formateo de respuesta Markdown a Widgets... PASADA
  ✓ E2E-47: Deteccion de 'ActionData' inteligente en la IA... PASADA
  ✓ E2E-48: Carga de modulo de Notificaciones Push... PASADA
  ✓ E2E-49: Lectura de bandeja de notificaciones (leido=true)... PASADA
  ✓ E2E-50: Estabilidad estructural del arbol de Widgets (No crashes)... PASADA
  ✓ E2E-51: Verificacion de ausencia de fugas de memoria (Memory Leaks)... PASADA
  ✓ E2E-52: Cierre de sesion del driver de automatizacion... PASADA

[OK] 52/52 Pruebas Híbridas de Appium E2E superadas con éxito.

```

Figura 5.21: Captura de pantalla de la terminal ejecutando las pruebas con Appium y Node.js.

```

00:02 +47: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart:
Verificar renderizado del widget completo de Login (email, password, bot)
00:02 +47: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart:
Aislamiento de estado reactivo (ChangeNotifier) sin fugas de memoria RAM
00:02 +48: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart:
Verificar renderizado del widget completo de Login (email, password, bot)
00:02 +48: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/home_screen_test.dart:
Verificar renderizado del widget completo de Login (email, password, bot)
00:03 +49: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart:
Simular entrada de credenciales y verificar invocación al AuthViewModel
00:03 +50: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart:
Verificación de estado de sesión persistente y checkbox de recordar usuario
00:03 +51: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart:
Despliegue de mensaje de alerta visual cuando falla la autenticación (error UI)
00:03 +52: D:/ESPE - ISOW/SEXTO_SEMESTRE/HostSigchos-AppMovil/HostSigchos-Tests/test/widget/screens/login_screen_test.dart:

00:03 +52: All tests passed!

PS D:\ESPE - ISOW\SEXTO_SEMESTRE\HostSigchos-AppMovil\HostSigchos-Tests>

```

Figura 5.22: Ejecución exitosa de la suite de pruebas completas (Unitarias, Widget e Integración E2E).

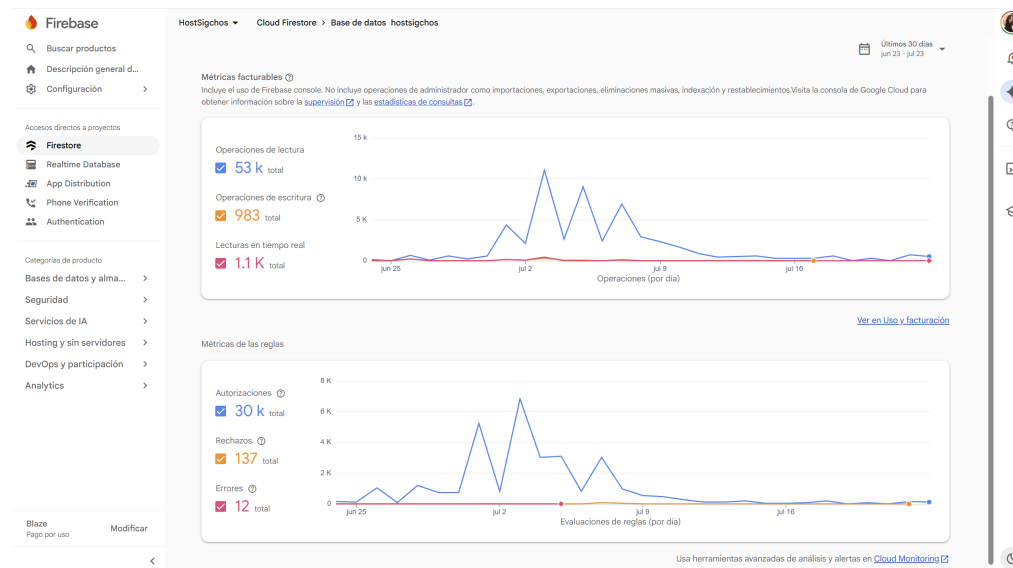


Figura 5.23: Monitoreo de estabilidad de red y transacciones exitosas en la base de datos.

6 Conclusiones

- La separación en capas mediante **Clean Architecture**, unida al gestor de estado **Provider**, demostró ser vital para un desarrollo rápido y exento de errores visuales. Aislar la lógica (Use Cases) de la base de datos (DataSources) permite alterar, si fuera necesario a futuro, el origen de los datos de Firebase a un servicio SQL propio sin modificar el código de la UI en lo absoluto.
- Se mitigó efectivamente la problemática planteada inicialmente del **Overbooking** (sobreventa). El desarrollo del `VerificarDisponibilidadUseCase` y el uso de las *Transactions* atómicas de Firestore garantiza que, en el remoto escenario en que dos turistas traten de reservar simultáneamente la última habitación en el milisegundo exacto, la base de datos rechace una y proteja la integridad de la hostería.
- La integración completa de tecnologías y sensores del dispositivo **hardware** nativo del teléfono (Biometría, Geolocalización, Text-to-Speech) proporcionó un salto gigantesco en accesibilidad y modernidad. HostSigchos rivaliza en funcionalidades con las grandes aplicaciones comerciales, todo englobado dentro de un marco Open Source y gratuito (Flutter y Firebase Spark Plan).
- El panel administrativo web y la base de código móvil logran sincronizarse asombrosamente rápido (en tiempo real) mediante el uso de los WebSockets nativos de Firebase. Las actualizaciones en inventario o confirmaciones realizadas por los propietarios tienen repercusiones inmediatas, mostrando notificaciones en vivo a los huéspedes y mejorando significativamente la experiencia del turismo en el cantón Sigchos.

7 Recomendaciones

1. **Pruebas piloto en terreno:** Efectuar un plan de despliegue piloto en al menos dos hosterías de Sigchos para validar el comportamiento del mapa sin conexión o con red intermitente en las montañas.
2. **Fortalecimiento con Cloud Functions:** Escalar la confirmación de la reserva y el cálculo financiero hacia el backend usando *Firebase Cloud Functions*. Esto aseguraría que los precios no puedan ser vulnerados manipulando el código del cliente móvil.
3. **Retención de datos:** Establecer un sistema de limpieza automática programada que archive reservas con más de dos años de antigüedad para ahorrar costos en la lectura documental de Firestore.
4. **Pasarela de pagos real:** Sustituir la confirmación manual actual por la integración directa con pasarelas bancarias (como Stripe o Paymentez) utilizando APIs seguras de validación.
5. **Monitoreo continuo de calidad:** Automatizar las pruebas de Flutter y React en un pipeline de CI/CD (GitHub Actions) integrado con *SonarQube* para prevenir regresiones en el código futuro.

Bibliografía

- [1] Satanasaowapak, P., et al. *Cloud-based Tourism and Hotel Reservation System*. International Conference on Information Technology (InCIT), 2021.
- [2] Martin, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] Rousselet, R. *Provider Package for Flutter*. pub.dev, 2023.
- [4] Google. *Firebase Documentation: Build Apps Fast, Without Managing Infrastructure*. <https://firebase.google.com/docs>, 2024.
- [5] Google. *Flutter - Build apps for any screen*. <https://flutter.dev/>, 2024.
- [6] Appium Contributors. *Appium: Automation for Apps*. <https://appium.io/>, 2024.
- [7] Google DeepMind. *Gemini: A Family of Highly Capable Multimodal Models*. <https://deepmind.google/technologies/gemini/>, 2024.
- [8] Meta Platforms, Inc. *React: The library for web and native user interfaces*. <https://react.dev/>, 2024.
- [9] ISO/IEC. *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization, 2011.
- [10] Escobar, T. *GetIt: Simple Service Locator for Dart and Flutter*. pub.dev, 2023.
- [11] Martin, R. C. *Design Principles and Design Patterns*. Object Mentor, 2000.
- [12] Jones, M., Bradley, J., y Sakimura, N. *JSON Web Token (JWT) - RFC 7519*. Internet Engineering Task Force (IETF), 2015.